

1 Introduction

2 Exploratory Data Analysis: Which of the possible target (outcome) variables should we use? (Continued)

3 Pre-processing and Feature Selection

4 Assessing Multicollinearity & Feature Redundancy

5 Unsupervised Learning Methods: Segmentation Analysis

6 Supervised Learning Methods

Table 10. Parsimonious OLS Regression Results.

7 Choose Your Own Adventure

References

Project 1: Biomedical & Clinical Informatics

Merrimack College DSE6630: Healthcare & Life Sciences Analytics

Lucia Adeola Alison Hartshorn

20 May 2025

1 Introduction

We will pick up where we left off in Demo 1 to prepare you for analysis. Although this is not going to be as exhaustive as we would like to do if we were doing a complete project, our goal is to see this project go from **messy data** → **prediction & interpretation**. After all, we need to give something to the patient advocacy watchdog that they can deploy!

1.1 Our Analysis Plan

Our goal is to see this analysis through to generate a predictive model that the advocacy group can use, and hopefully it will be a decent one. To ensure that it stands the best chance of being a good predictive model, our analysis plan includes all the key steps in the data science life cycle/project flow, including: **cleaning, exploration, feature selection, pre-processing, unsupervised machine learning exploration, and supervised modeling**. Whew!

Our specific plan is to perform two big analyses in the hopes of having a nice deliverable to present to our client. The first will be a **segmentation analysis**, or clustering analysis, which will attempt to group our hospitals together on the basis of their attributes and their pneumonia-related readmissions. This will be done using a combination of PCA and k -means. Our second analysis will be to perform supervised, predictive analysis using an elastic net, which conveniently combines a penalized regression with variable selection so we can point our clients down a path of hospital attributes to focus on, as well as a model they can use to make future predictions.

1.2 Your objective.

Your object is to work through this Project 1. You will have 31 Questions to answer as you work through this, just as with Project 1. Then, you will be able to choose your own adventure again to perform your own **clinical informatics** analysis:

Adventure 1.

[Minimal coding, focus is on understanding cleaning, pre-processing, and the ML analyses we performed here.] Choose ONE other condition (anything other than pneumonia) and run through the analyses again, using our same target variable. You will update your question, hypotheses, and predictions, but will be able to re-use all of the other code with very minor

modifications. Credit will be based more on interpretation and not on coding. See more information on Adventure 1 here.

Adventure 2.

[More coding, but not as much, with a focus on the steps we've undertaken here and extending it to a new question & condition.] Choose a slightly more complicated analysis to undertake, for example, focusing on surgical interventions (HIP-KNEE & CABG) or heart-related conditions (HF, AMI, & CABG). The advantage of this analysis is that instead of looking at a single condition, you will be looking at the **average** hospital readmissions rate for the conditions you select. This might also enable us to say something far more informative to a subset of patients (e.g., those considering undergoing surgical interventions). See more information on Adventure 2 here.

Adventure 3.

[Most coding, but no cleaning or pre-processing, with an emphasis entirely on the machine learning analyses.] Continue with the pneumonia-related data that you have here, completely cleaned and pre-processed, and do two more unsupervised OR supervised analyses. For example, you could do another kNN using the k we identify in our Segmentation Analysis here, or you could do a random forest to account for any possible non-linearity. You could even choose to do a Ridge and LASSO regression and compare those analyses to what we get out of an Elastic Net. Any of these are fine, but you must **explain your decision** in your submission. See more information on Adventure 3 here.

1.3 A Fork in the Road!

This is the first of your choices in your adventure, and what you choose here could either lead to success or... **sudden death** (dum, dum, dummmm...). Choose wisely, young data scientists!

Option 1.3.1

Load data from the end of Demo 1, which includes the ordinal and frequency encoding you did (plus a little bonus cleaning to help).

Load my version of the fully cleaned, encoded, & ready-to-proceed dataset from the end of Demo 1. These data are not yet ready for analysis (!) but we would be picking up with that here.

```
## Uncomment if this is your choice:  
load("FY2024_data_files/pneumoniaAnalyzeFullyEncoded2024.Rdata")  
dat2Analyze <- pneumoniaAnalyzeFullyEncoded
```

Option 1.3.2

Load data from Demo 1 without ANY encoding, but all other tidying the same.

```
## Comment out if this is NOT your choice:  
#load("FY2024_data_files/pneumoniaAnalyzeNoEncoding2024.Rdata")  
#dat2Analyze <- pneumoniaAnalyzeNoEncoding
```

Question 1: [1 point]

Which dataset do you choose and why?

Explain your selection here. ##We chose Option 1.3.1, the fully encoded dataset, because many of the predictors in this hospital dataset are categorical and machine learning analyses generally require numerical inputs rather than text-based categories. Since the ordinal and frequency encoding from Demo 1 has already been completed, this option provides a more efficient and reproducible starting point without requiring us to repeat preprocessing work that has already been done. While choosing Option 1.3.2 would have given us more opportunity to revisit and validate the encoding process ourselves, it would also increase the likelihood of introducing inconsistencies or coding mistakes when recreating preprocessing steps. Because our goal is to continue into feature selection, preprocessing, clustering, and predictive modeling, we felt Option 1.3.1 allowed for a more streamlined workflow while still recognizing that additional preprocessing will be needed before analysis.

1.4 Helper code: Using R like an object-oriented language

Many of you still may be early enough in your programming journeys that you haven't yet learned how to fully leverage the **object-oriented** nature of programming languages like Python. R is not *technically* an object-oriented language in the same way, but we can hack it to function like one. That's exactly what we are going to do here by loading a **helper function** directly from the source code.

We went through all the trouble in the Demo to perform encoding and now we get to use it! You will use the `source()` function to load a helper function that I have written, using my answer to the ordinal encoding question of the Demo, so that you can do all your **ordinal** encoding in a single line of code. Neat, huh?

BUT WHY are we doing the encoding now if we just chose to bring in non-encoded data? Because we can do **ordinal encoding** before or after data partitioning with impunity. It does not introduce data leakage to do so here, so we can do it in either order. This means that we can do it now simply for convenience.

NOTE: If you chose Option 1, the fully-encoded dataset, you can still run this chunk because there is a conditional in here that will not run your dataset since it is already encoded! But still make sure you understand the idea of source code because we will keep using it throughout the course!

```
## This calls the code in the associated .R file
source(file = "doOrdinalEncoding.R")

## This checks to see if you loaded the already encoded dataset
if(!exists("pneumoniaAnalyzeFullyEncoded")) {
  ## This sets a list of PATTERNS to match for the columns to ordinal encode
  encodeList <- c("ComparedToNational_",
                  "PaymentCategory",
                  "ValueOfCareCategory",
                  "Score_Emergency department volume")

  ## This runs the function, which takes 3 arguments
  ## Open the source code to see what each argument does!
  dat2Analyze <- doOrdinalEncoding(df = dat2Analyze,
                                   encodeList = encodeList,
                                   quiet = FALSE)
}
```

2 Exploratory Data Analysis: Which of the possible target (outcome) variables should we use? (Continued)

NOTE: What follows is my (long-winded) answer to **Question 19** from the Demo.

2.1 Overview of our options

I am going to quickly break down the differences between the possible targets so we can make sure we really understand **what** they are measuring to allow us to make our most-informed selection of a target. Note: More information was found in this PDF (https://qualitynet.cms.gov/files/5d0d3ae5764be766b0104c37?filename=Pneumo_ReadmMeasMethod.pdf). **This is really important for helping us choose the right one!**

Notice that I am also including a variable we didn't look at previously called `ComparedToNational_Hospital` return days for pneumonia patients. This is because this is our best assessment of how the hospital is performing with regard to pneumonia readmissions relative to a national average. However, even though I'm including it in the table, I think it's better to think of it as a **solid predictor**. It's telling us how **length of stay (LOS)** in the hospital after pneumonia readmission stacks up compared to the national average. After a hospital readmission, it's generally better to have *fewer* return days rather than more because it suggests better quality of care, more timely follow-up, and a greater chance of the patient recovering well at home. Since these are readmissions, having *more* days than average could also suggest that the hospital missed the severity or didn't properly plan for the patient's recovery at home and released them too early.

You may be wondering - could we use it as a target? Sure - but it's not for the question we were posed by our stakeholder! Note that, if we did, we would need to perform a multinomial classification analysis. So, instead, let's use this category to help us contextualize our hospitals as we choose between Predicted, Observed, and Expected readmission rates, as well as Excess Readmission Ratio.

Table 1. List of possible target variables from the pneumonia-related hospital readmission data. Which one to choose?

Possible Target	Description
<code>ComparedToNational_Hospital</code> return days for pneumonia patients	Hospital return days measures the number of days patients spent back in the hospital (in the emergency department, under observation, or in an inpatient unit) within 30 days after they were first treated and released for pneumonia. Reported as compared to the national average, such that 'below average' is better and 'above average' is worse.
<code>ExpectedReadmissionRate</code>	The expected number of readmissions in each hospital is estimated using its patient mix and an average hospital-specific intercept. It is thus indirectly standardized to other hospitals with similar case and patient mixes.
<code>PredictedReadmissionRate</code>	The number of readmissions within 30 days predicted based on the hospital's performance with its observed case mix. The predicted number of readmissions is estimated using a hospital-specific intercept, and is intended to reflect the annual expected performance of the hospital given its historical case and patient mix and performance.
<code>ExcessReadmissionRatio</code>	The ratio of the predicted readmission rate to the expected readmission rate, based on an average hospital with similar patients. Performance is compared against a ratio of one, such that below one is better and above one is worse in terms of readmission rates.

Possible Target	Description
observed_readmission_rate	Our calculation of the observed number of pneumonia-related readmissions within 30 days found by dividing the number of readmissions observed for the hospital during the given period by the number of discharges for that same period, and multiplied by 100 to put it on the same scale as the Predicted and Expected Readmission Rates. It reflects a true observation as reported by the hospital during that period, but is not adjusted for case mixes or prior information for that hospital. Thus, this is a crude, unadjusted value.

2.2 Summary statistics of the possible targets

Now, I know I didn’t ask you to do it, but I will actually begin my assessment of which target is most appropriate by constructing a quick summary table. In my assessment of what is an appropriate target,

Table 2. Possible target measure summaries

Characteristic	N = 4,816 ¹
National Comparison Return Hospital Days	
Better than average	402 / 4,816 (8.3%)
Same as average	2,285 / 4,816 (47%)
Worse than average	827 / 4,816 (17%)
Unknown	1,302 / 4,816 (27%)
Predicted Readmission Rate	16.46 (1.93), 16.27
(Missing)	2,090
Observed Readmission Rate	17.1 (3.7), 16.9
(Missing)	2,603
ExcessReadmissionRatio	1.00 (0.06), 1.00
(Missing)	2,090
Expected Readmission Rate	16.43 (1.51), 16.39
(Missing)	2,090
¹ n / N (%); Mean (SD), Median	

We can see right away that we lose nearly **600 more data points** if we opt to use our calculated `observed_readmission_ratio` (NOTE: nearly 500 if using the 2025 FY data), which is going to be a by-product of the fact that the `Predicted` and `Expected` readmission ratios are using prior information and information about patient mixes to generate these values. Our `observed_readmission_ratio` is crude and unadjusted any way - which could present problems when it comes to true comparability. **So, off the bat it feels as if the constructed target is possibly not a good choice despite its simplicity for stakeholders to understand.**

2.3 Comparing the four possible metrics to ComparedToNational_Hospital return days for pneumonia patients

Although ComparedToNational_Hospital return days for pneumonia patients is very compelling, it's better as a **predictor** rather than a target because it gets at an aspect of hospital performance: once readmitted due to pneumonia, how long (as compared to the national average) are patients spending in the hospital? This means it makes a really nice predictor to use to *compare our four possible choices for pneumonia-related hospital readmissions*. We are looking for a clear separation in our ideal target between hospitals that have lower readmission rates AND also tend to do better than the national average at healing their readmitted patients!

So, this makes a really nice variable I could choose to **facet** plots, which means creating sub-plots that show up as panels. The thing is, if I know I want to **facet** plots, the first thing I have to do is reshape the data I want from *wide* into *long* format using, for example, the `pivot_longer()` function (the complement of `pivot_wider()` that we used in the Demo).

I don't have to store it as a separate data frame (i.e., I can just pipe it directly into `ggplot()` !) but here I am choosing to so we can more easily make sure it's reshaped correctly along the way. Thus, I am making a long-format version of the possible target variables called `longTargets` so that I can better decide which target to choose.

I mentioned violin and density plots specifically in Question 19 of Demo 1, as well as faceting. I am going to show you two ways to explore these data with these types of plots and you can decide which one(s) you find most informative here. You also may have thought of a better way to display the data too!

2.3.1 Example with `geom_violin()` :

Here I am leaving the NA data (which I relabelled as `Unknown`) to get a sense for the degree of missingness.

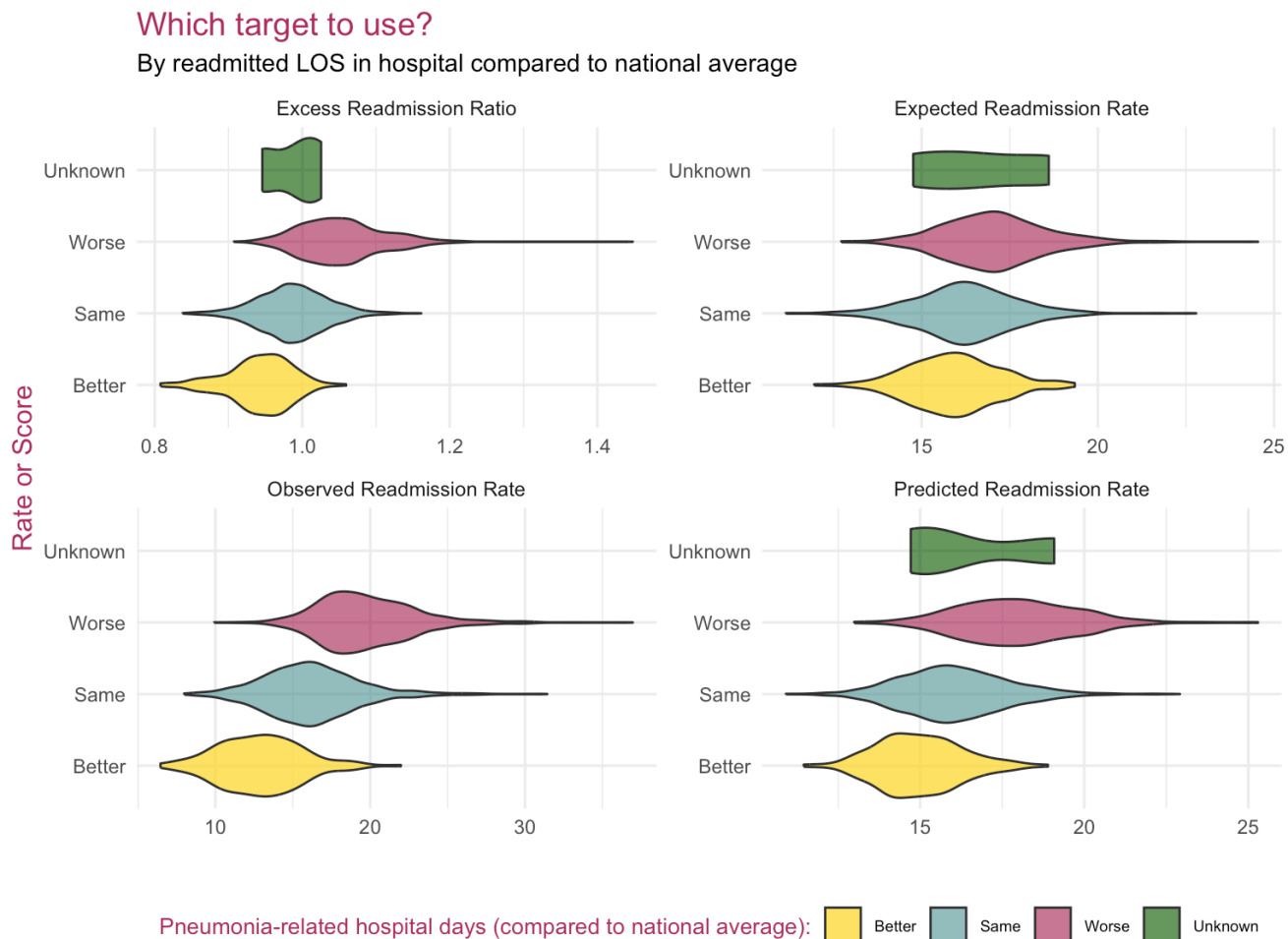


Figure 1. Readmission Rates vs. readmitted hospital LOS

Question 2A: [0.5 points]

If you have never encountered violin plots (https://en.wikipedia.org/wiki/Violin_plot) before, they are honestly a wonderful way to assess the **symmetry** and **skew** of a distribution while more easily enabling comparisons between groups. They're effectively a hybrid between a histogram and a boxplot, and we can even opt to overlay the boxplot quartiles in the plots as well, which is particularly handy!

Your task is to do just that by adding `geom_boxplot(width = 0.1)` (yes, just that one line!) in the appropriate place. Also change `include = FALSE` to `include = TRUE` in the R chunk header so that it shows up when you knit!

Make sure to comment your line to call attention to it:

Which target to use?

By readmitted LOS in hospital compared to national average

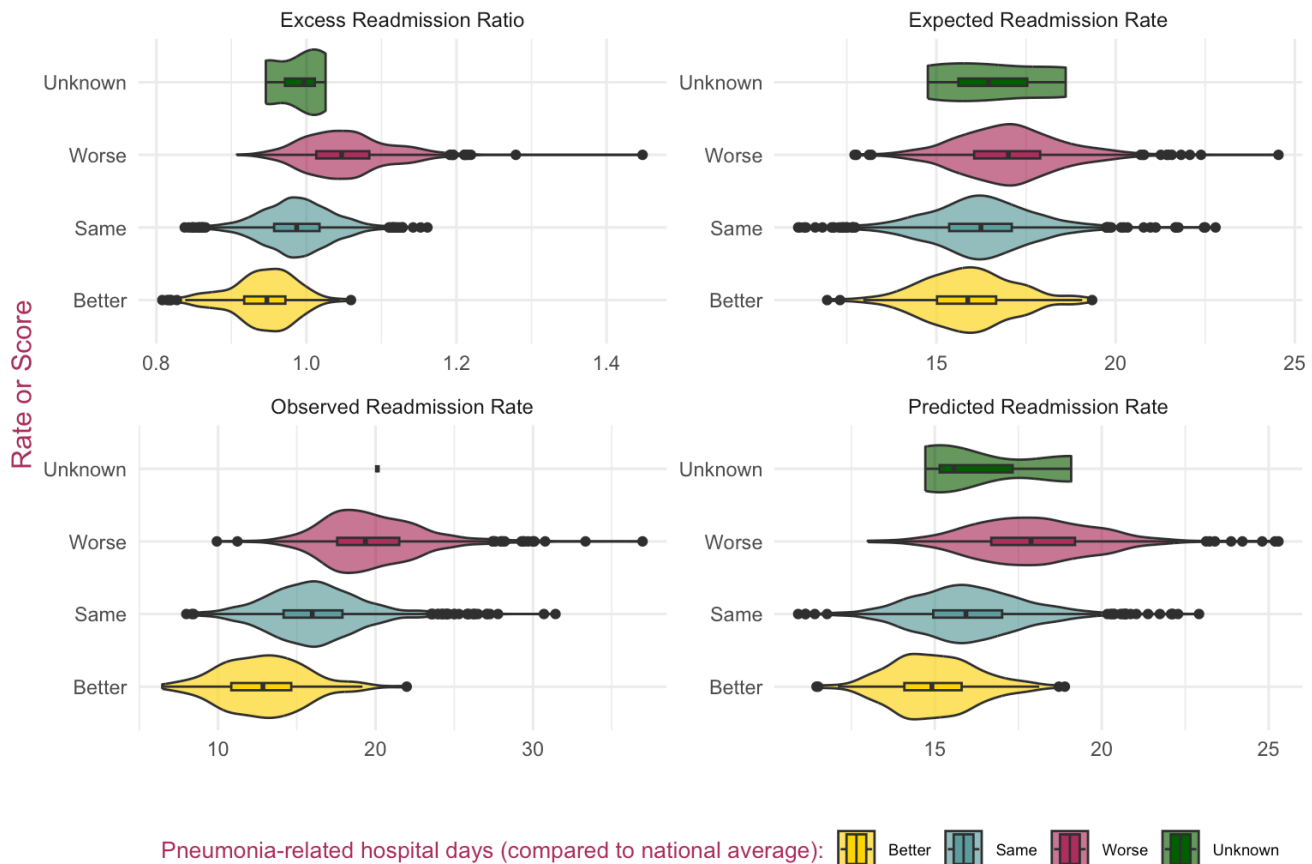


Figure 2. Readmission Rates vs. readmitted hospital LOS

Question 2B: [0.5 points]

Now that you can compare these groups more easily, what we're looking for is a balance between **clear difference** between hospital ordinality relative to the national average (i.e., "Are better hospitals really better in pneumonia readmission rates?").

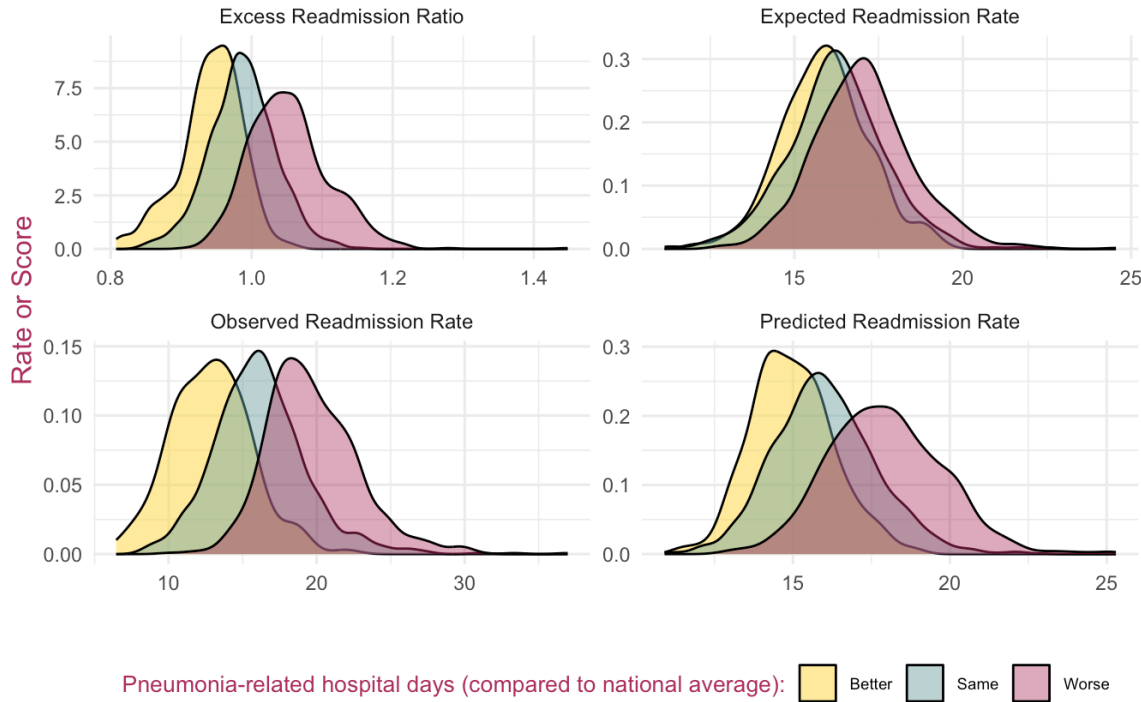
Which target(s) appear the most distinct to you? > Your answer here. ##After adding the boxplots, Observed Readmission Rate and Predicted Readmission Rate appeared to be the most distinct target variables because they showed clearer differences between hospital groups. In comparison, Expected Readmission Rate and Excess Readmission Ratio had much more overlap, with similar medians and quartile ranges across better, same, worse, and unknown hospitals, making differences harder to see. Between the two stronger options, Predicted Readmission Rate appears to show the clearest hospital ordinality, since better hospitals had lower values, same hospitals were in the middle, and worse hospitals have the highest values with less overlap between groups.

2.3.2 Example with `geom_density()`:

We could also choose to overlay density plots instead of violin or boxplots, as this can sometimes help us see subtler separation of groups than we can see otherwise. This time, I am also going to ignore the “Unknown” (i.e., missing) class of hospitals to prevent things from getting too messy!

Which target to use?

By readmitted LOS in hospital compared to national average



Question 3: [0.5 points]

Does this view confirm or change your decision from Question 3?

Your answer here. ##Looking at the density plots, this confirms our decision from Question 2B. The Predicted Readmission Rate still appears to be the most distinct target because there is clearer separation between better, same, and worse hospitals with less overlap between the groups. At first, we thought Observed Readmission Rate also showed strong distinction in the violin plot, but after looking at the density plot, the overlap between hospital groups became more noticeable. In comparison, Expected Readmission Rate and Excess Readmission Ratio still seem much more overlapping and compressed, making it harder to clearly see differences between hospital categories.

2.4 What is the relationship of pneumonia-related hospital readmissions to the hospital performance metrics?

Something else we could do to help us make a decision is compare across predictors to look for the most striking relationships among the proposed target (readmission rate), how well compared to national average hospitals are performing AFTER readmitting pneumonia patients (how long after readmission they are in the hospital), and relationships among other hospital-performance predictors, like Medicaid spending or pneumonia-related deaths.

So, to do this, I'm going to make another long-format dataset and use this to make a bunch of faceted scatterplots where I put one of the readmission rates (let's started with Predicted Readmissions) and look for strength in relationships. Note that doing it this way, through plotting, is a **heuristic way** of choosing a target variable and it isn't the only way to do it!

Question 4: [1 point]

Change the **y-axis** out for _each of the four readmission rates. Predicted Readmission Rates has been started for you. Re-run the chunk and compare the results. **FOR FULL CREDIT** you must leave the graph on the choice you feel most strongly about AND update the title of the graph with which of the four possible readmission rates you chose. Explain below why you chose this one.

Your answer here. ##After comparing all four readmission targets, we decided to use Predicted Readmission Rate because it was the easiest to interpret visually and showed the clearest separation between better, same, and worse hospitals. Compared to Expected Readmission Rate and Excess Readmission Ratio, there was less overlap between the groups, which made the trends across hospital performance measures easier to see. While Observed Readmission Rate was more interpretable than expected and excess, it still appeared more spread out and visually noisier than predicted readmission. Looking across the performance metrics, several relationships were easier to identify using predicted readmission. For example, hospitals with higher Medicare spending per patient appeared to have higher predicted readmission rates, especially among worse-performing hospitals, which may reflect more medically complex or higher-risk patient populations. We also noticed a negative relationship for healthcare worker influenza vaccination measures, where stronger vaccination-related performance appeared to be associated with lower predicted readmission. In addition, pneumonia death rate showed a slight negative trend, although the relationship appeared weaker than the patterns observed for Medicare spending. Overall, predicted readmission rate produced the clearest and easiest-to-interpret relationships across hospital performance metrics, making it the strongest choice for our analysis.

HINT 1: What does higher medicare spending per patient tell you about the underlying population of a hospital?

HINT 2: Venous thrombolisms and deep vein thromboses (DVTs) are often “sneakier” conditions that have “silent” symptoms until the patient is in crisis. What would having higher scores at providing prophylaxis for these conditions potentially suggest about a hospital’s performance?

Relationship of Predicted Readmission Rate for pneumonia-related hospi readmissions to each performance metric

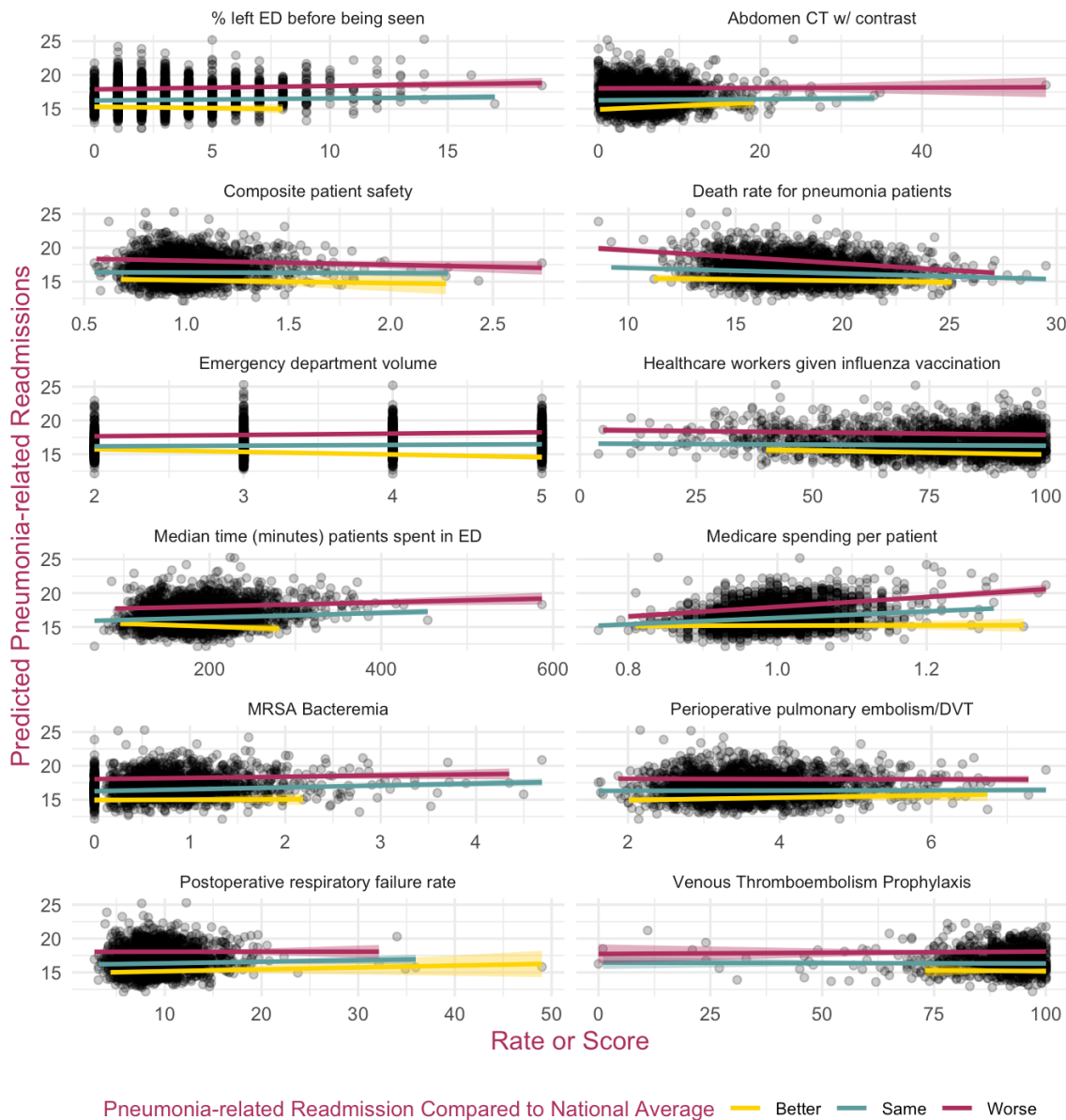


Figure 4. Readmission rate by hospital performance and readmitted patient LOS

HINT 3: Here's how I would interpret the **relationship between PREDICTED pneumonia readmissions and pneumonia death rates, looking at whether hospitals performed better, same, or worse in terms of length-of-stay after readmission.** What I notice immediately is a moderate slope for hospitals with worse readmitted lengths of stay (LOS) [maroon line] compared to same [blue line] and especially better [yellow line]. This relationship is negative, implying that as the pneumonia-related death rate increases, readmissions **goes down**. Oh! So what this implies is that hospitals with higher deaths from pneumonia tend to have lower readmissions - and this is especially true for the ones who are have longer (worse than average) readmitted patient stays. In other words - these hospitals are probably releasing their pneumonia patients too soon from their first hospitalization, leading to higher readmissions, longer readmitted stays, and even higher pneumonia-related deaths!

2.5 What is the relationship of pneumonia-related hospital readmissions to the patient satisfaction metrics?

Now let's do a similar analysis but for the `HcahpsLinear...` variables. The **HCAHPS (Hospital Consumer Assessment of Healthcare Providers and Systems)** is a standardized survey that measures adult inpatient perceptions of hospital care. It's a national, publicly reported survey mandated by the Centers for Medicare & Medicaid Services (CMS). Because it

focuses on aspects of hospital care that are important to patients, it is often used to improve patient satisfaction and hospital quality.

Question 5: [1 point]

Again, your task is to change the **y-axis** out for each of the four readmission rates. Predicted Readmission Rates has been started for you. Re-run the chunk and compare the results. **FOR FULL CREDIT** you must leave the graph on the choice you feel most strongly about AND update the title of the graph with which of the four possible readmission rates you chose. Explain below why you chose this one.

Your answer here. ##After comparing all four readmission targets, we decided to again select Predicted Readmission Rate because it was the easiest to interpret visually and showed the clearest relationships across the patient satisfaction measures. Compared to Expected Readmission Rate and Excess Readmission Ratio, there was less overlap between hospital groups, which made the patterns much easier to see. While Observed Readmission Rate was also easier to interpret than expected and excess, it still looked more spread out and visually noisier than predicted readmission. Looking across the patient satisfaction measures, we noticed that variables related to communication, especially doctor communication, communication about medicines, discharge information, and staff responsiveness, showed clearer negative relationships with readmission. In other words, as patient satisfaction with communication increased, readmission rates generally decreased. This makes sense because better communication may help patients better understand medications, discharge instructions, and follow-up care, which could reduce the chances of being readmitted. On the other hand, cleanliness did not seem to have as strong of a relationship with readmission, which suggests that communication-related factors may have a stronger relationship with patient outcomes in this analysis than environmental satisfaction measures.

HINT 1: What does higher nurse and doctor communication mean for improved patient outcomes, including readmission?

HINT 2: Why might cleanliness not have much effect here even though pneumonia is a communicable disease?

Relationship of Predicted Readmission Rate for pneumonia-related hospi readmissions to each patient satisfaction metric

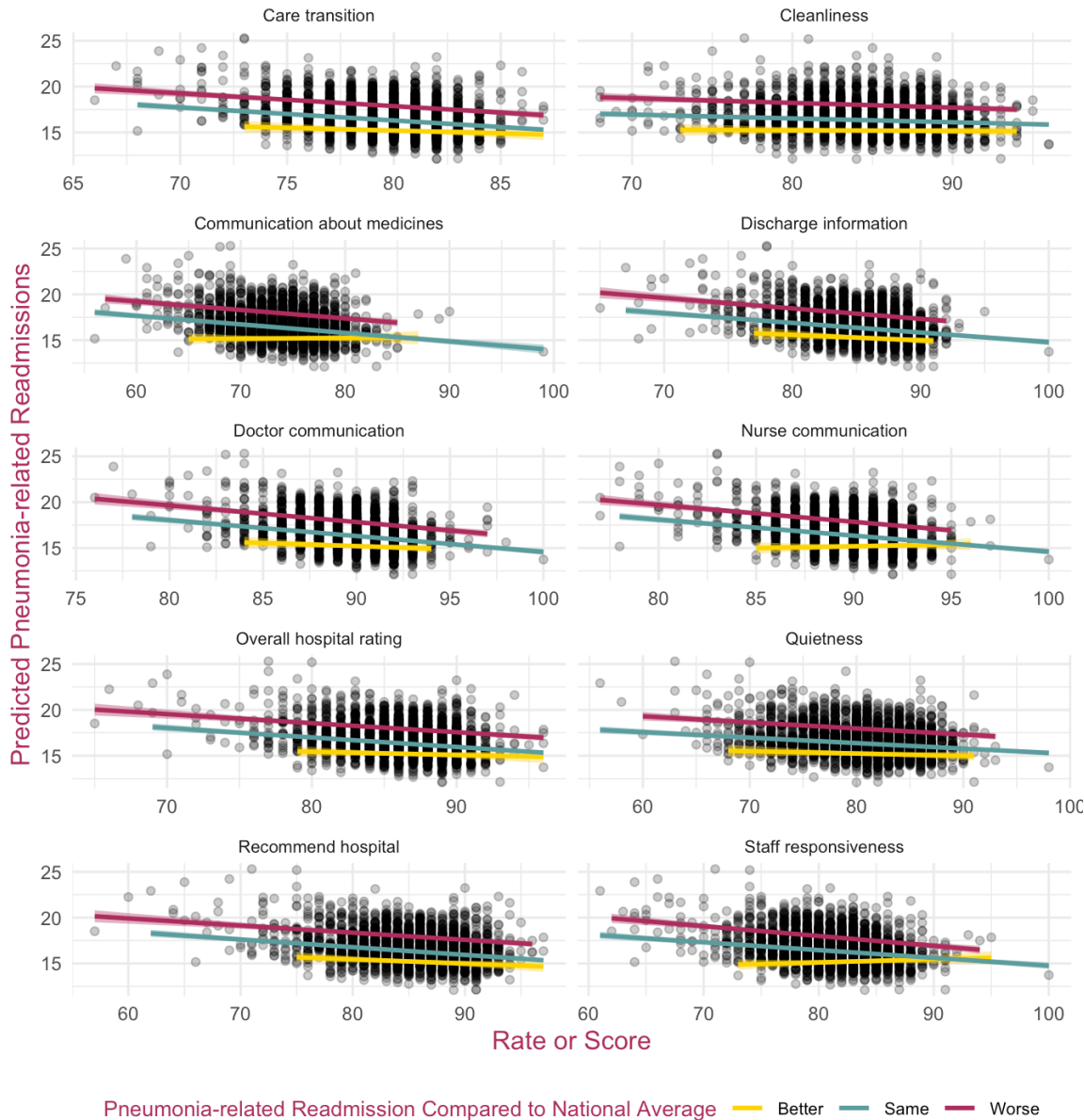


Figure 5. Readmission rate by patient satisfaction and readmitted patient LOS

HINT 3: Here's how I would interpret the relationship between **PREDICTED** pneumonia readmissions and overall hospital rating, looking at whether hospitals performed better, same, or worse in terms of length-of-stay after readmission. What I notice immediately is a pretty substantial negative slope for hospitals with worse readmitted lengths of stay (LOS) [maroon line], moderate negative slope for hospitals with the same readmitted LOS [blue line], and slightly decreasing slope for hospitals with better readmitted LOS better [yellow line]. So what this implies is that hospitals with high ratings tend to have lower pneumonia-related readmissions, even among the hospitals with worse readmitted length-of-stays. This relationship is strongest when hospitals have longer LOS; maybe having a long readmission LOS stay **actually implies a form of quality**, like "Well, we got it wrong the first time, let's make darn sure we get it right the second?" But hospitals that were already getting it pretty right don't have as steep of slope because they were already doing well at readmissions to begin with, hence the almost flat line.

Question 6: [2 points]

What relationships do you find most compelling from these scatterplots and why? Also, name one outstanding question you have at this point, whether it is about the data, the data collection, or something more speculative.

Your answer here. ##The relationships we found most compelling were doctor communication, nurse communication, staff responsiveness, and Medicare spending per patient. For the communication-related measures, we noticed that as communication scores increased, readmission generally decreased, suggesting that hospitals with stronger communication may have lower pneumonia-related readmission. This makes sense because better communication may help patients better understand medications, discharge instructions, and follow-up care. We also found Medicare spending per patient interesting because as spending increased, readmission also appeared to increase across hospital groups, which may reflect hospitals treating more medically complex or higher-risk patient populations. One question we still have is how much patient health complexity and hospital location influence these relationships. For example, hospitals in major metropolitan areas may see more medically complex patients or larger patient volumes compared to hospitals in suburban or less populated areas, which could affect readmission rates independent of communication quality or Medicare spending. Because of this, we do not think communication or spending alone fully explains hospital quality or readmission outcomes.

Although there is certainly more EDA we could do, we are going to wrap that up here for the sake of moving on - but, as always, I encourage you to think about or even explore what else you would like to know about your data that we did not do!

2.5 Final remarks on target selection.

Going forward, I have selected `PredictedReadmissionRate` as our target. In a more thorough analysis, we would likely want to consider:

1. Circling back to our stakeholder to try to refine our question or target using domain knowledge
2. Perform experiments with our models to find the best predictive model since that is our ultimate goal.

However, given that we do not have the option or time for that, let me explain my choice. It's the **best balance** of distinguishing hospitals with better and worse readmitted hospital stays, while still providing an **adjusted, hospital-specific** measure. Adjusted values will generally always be a better choice than the crude, even if our crude measure captures something interesting about the data, as it did here, or if it's easier for our stakeholder to comprehend. Our crude measure, `observed_readmission_rate` did the **best job** of recapitulating the readmitted hospital LOS (ComparedToNational_Hospital return days for pneumonia patients), but if that's really our focus, wouldn't it then be better to switch from a readmission rate to that as our target? Probably.

If you still feel unsure about what is the best choice, you're not alone - so do I! This is where, because we are working with an **imagined stakeholder**, we must make the best decision we can given the question we defined in the Demo. I would defend my choice of `Predicted Readmission Rate` for three key reasons:

- It has been adjusted to be hospital-specific, using a hospital-specific intercept and historic data about the hospitals' performances
- We don't lose nearly 600 data points to missingness.
- Our imagined stakeholder, and the question we set, was about **readmission rate** so even though **readmitted return hospital days** might also be a similarly viable candidate, it doesn't reflect our imagined stakeholder's request. Note that the `ExpectedReadmissionRate` is standardized for similar hospitals, and may not truly reflect a given hospital; the `ExcessReadmissionRatio`, while informative, was the least likely to reflect the distinctiveness in hospital return days, which is still of interest to us.

But remember that what matters most is that we are **matching our question**.

Question 7: [1 point]

Do you agree with my choice of target? Why or why not? (You are free to disagree!)

##At first, based on what we saw in Demo 1, we would have selected Observed Readmission Rate because it represented the actual readmission outcome observed in hospitals, so at first it felt like the most direct and meaningful thing to model. It also seemed to show strong spread, variability, and separation across hospital groups. However, once we loaded the fully encoded dataset and started the exploratory analysis, we reconsidered that choice after seeing how much missingness existed in the observed readmission variable. Since such a large portion of the data was missing, it made us question how reliable it would be to use as our target variable.

After looking at the violin plots and scatterplots, we noticed that Observed Readmission Rate and Predicted Readmission Rate performed pretty similarly in terms of spread, variability, and showing relationships across variables. However, observed readmission looked slightly noisier in some of the visualizations, which we think may be related to the amount of missing data. We also agreed with the idea that an adjusted measure may be a better choice than a crude one in this case. Since Observed Readmission Rate is more of a crude measure, while Predicted Readmission Rate accounts for adjustment and showed many of the same useful patterns with fewer missingness concerns, we ultimately agreed with using Predicted Readmission Rate as the stronger and more practical target variable for our analysis.

2.6 Bias and Limitation

Question 8: [2 points]

All studies have bias (introduced or systemic error) and limitations (failure to fully explain something). We could go more deeply into these concepts, but for now I want you to take a stab and just brainstorming either ways we might have bias OR the limitations of using this target variable. You do not need to answer both, but you're welcome to do so. You can read more about bias here (<https://www.ncbi.nlm.nih.gov/books/NBK574513/>) or see some deeper explanations of limitations here (<https://www.aje.com/arc/how-to-write-limitations-of-the-study/>). You do not need to write a lot; just 2-3 sentences. I am simply asking you to pause and reflect on our choices here before we proceed.

Your answer here. ##One limitation of using Predicted Readmission Rate is that, even though it reduced missingness and gave clearer patterns in our analysis, it is still an adjusted or predicted measure rather than the actual observed hospital outcome. Because of this, it may not fully capture what is actually happening at a specific hospital or account for other factors that may be region, location, or volume-specific. For example, hospitals in major metropolitan areas or hospitals treating larger numbers of medically complex patients may naturally appear to perform worse on readmission, even when they are providing strong care, simply because they are operating under very different conditions than smaller suburban or rural hospitals. This also introduces potential bias because hospitals are not all serving the same patient populations or operating under the same demands, therefore differences in readmission may sometimes reflect patient complexity or hospital volume rather than actual hospital quality alone.

3 Pre-processing and Feature Selection

The time has finally come to officially move **back** into pre-processing and feature selection so that we can finally build some models. I want you to take note of the order in which I'm doing things here; the flow can change from project to project, but generally it's a good idea to get your dataset to a state of **manually curated features** before making your data partition.

3.1 Further feature selection and refinement.

3.1.1 Dropping the targets we aren't planning to use. [Manual Method]

It's time to say goodbye to the features we know would be far too redundant or non-informative given our target of `PredictedReadmissionRate`. We don't need any of the `Sample_` columns, for example; these are merely sample sizes. Useful if we were doing standardization, but not necessary for us right now. We also are dropping the other possible target features that are too redundant with `PredictedReadmissionRate`.

Question 9A: [0.5 points]

Add your comments to the code as indicated to explain what I was doing at each of the steps.

Your answer in the comments.

```
dat2Analyze <- dat2Analyze %>%
  ##Drop any other possible readmission target variables and sample-size/count variables
  ## since we have selected PredictedReadmissionRate as our final target
  select(-ExpectedReadmissionRate,
        -ExcessReadmissionRatio,
        -observed_readmission_rate,
        -contains(c("Sample_", "NumberOfPatients")),
        -NumberSurveysCompleted) %>%
  ##Rename long variable names into shorter, cleaner names so the dataset is easier to read/interpret
  rename(`Median time (minutes) patients spent in ED` =
        `Score_Average (median) time patients spent in the emergency department before leaving from the visit A lower number of minutes is better`,
        `Composite patient safety` = `Score_CMS Medicare PSI 90: Patient safety and adverse events composite`,
        `ComparedToNational_Composite patient safety` = `ComparedToNational_CMS Medicare PSI 90: Patient safety and adverse events composite`) %>%
  ##Move the target variable to the first column so it's easier to find and keep track of during preprocessing
  select(PredictedReadmissionRate, everything())

## Clean up column names by removing repeated prefixes/suffixes
colnames(dat2Analyze) <- gsub('Score_|HcahpsLinearMeanValue_|_Payment', '', colnames(dat2Analyze))
```

3.1.2 Dropping any features with near-zero variance. [Automated Method]

Next, we are going to take advantage of the `nearZeroVar()` function that is part of the `caret` package. We will drop any of the columns with near-zero variance. This is a way of automating (and thus standardizing) the process for choosing to drop non-informative columns due to zero or nearly zero variance.

Question 9B: [0.5 points]

If you are not familiar yet with the concept of **zero or near-zero variance**, do a little digging. Based on what you read, try to explain for your naive advocacy group stakeholders, why it's important to remove the features with little variance.

Your answer here. ##Features with zero or near-zero variance are not very useful because they do not help distinguish one hospital from another. If almost every hospital has the same value for a variable, that variable will not add much information to the model and may only make the analysis more complicated or noisy. Removing these features helps keep the dataset cleaner and allows the model to focus on variables that actually show meaningful differences across hospitals.

Table 3. Columns dropped due to near-zero variance.

Variables Dropped

ComparedToNational_Postoperative respiratory failure rate

ComparedToNational_Periooperative pulmonary embolism or deep vein thrombosis rate

Take note of the number of features here.

Question 9C: [0.5 points]

You may or may not have noticed that I moved `FacilityID` to our rownames in the datasets you loaded today. Why did I choose to do that? Why do I want to make sure unique identifiers, like `FacilityID`, are out of the dataframe before I either (a) run `nearZeroVar()` or (b) proceed with the rest of the analysis?

Your answer here. ##FacilityID was moved to the row names because it is a unique identifier for each hospital, not a feature that should be used to predict readmission. If it stays in the dataframe, models may treat it like a meaningful variable even though it is actually a label. We also want it removed before running `nearZeroVar()` and proceed with the rest of the analysis because unique IDs do not add useful predictive information and could interfere with feature selection and modeling.

Question 9D: [1 point]

Do some research on, or lean into your previous knowledge, about the **curse of dimensionality** or the **big-P, little-N** ($n \gg P$) **problem**. Using whatever you find or remember, can you justify why we likely do NOT have a **big-P, little-N** ($n \gg P$) **problem** here? Make sure to explain.

Your answer here. ##We likely do not have a big-P, little-N problem because a curse of dimensionality problem usually happens when there are too many predictors compared to the amount of actual data. In our case, we have far more hospital observations than predictor variables, meaning we have enough data for the model to learn meaningful relationships instead of memorizing random noise. Because we have more data than predictors, our model is less likely to overfit and more likely to learn actual patterns between the predictors and hospital readmission outcomes. We also reduced unnecessary predictors during preprocessing by removing near-zero variance variables, including Payment category for pneumonia patients, ComparedToNational_Postoperative respiratory failure rate, and ComparedToNational_Periooperative pulmonary embolism or deep vein thrombosis rate, which further reduced noise and the chance of overfitting.

3.2 Assess missingness & devise an imputation plan

We are going to deal with imputation in a rather quick and elegant way that requires minimal effort from us using a model-based approach with `missForest`. You may recall from lecture that I said we are choosing a model-based approach *without uncertainty* because our ultimate goal is to create a deployable model for our stakeholder.

Question 10: [1 point]

Write some code that will quickly tally the number of columns in the dataset that contain missing values. You will find `colSums()` and `is.na()` functions likely useful here. **What percent of the features have at least some level of missingness?**

```
colSums(is.na(dat2Analyze))
```

```

## PredictedReadmissionRate
## 2090
## MRSA Bacteremia
## 3083
## Payment for pneumonia patients
## 1481
## Abdomen CT Use of Contrast Material
## 889
## Death rate for pneumonia patients
## 1302
## Postoperative respiratory failure rate
## 2213
## Perioperative pulmonary embolism or deep vein thrombosis rate
## 1881
## Composite patient safety
## 1805
## Medicare spending per patient
## 1830
## Healthcare workers given influenza vaccination
## 676
## Median time (minutes) patients spent in ED
## 749
## Left before being seen
## 975
## Venous Thromboembolism Prophylaxis
## 2600
## Nurse communication
## 1561
## Doctor communication
## 1561
## Staff responsiveness
## 1561
## Communication about medicines
## 1561
## Discharge information
## 1561
## Care transition
## 1561
## Cleanliness
## 1561
## Quietness
## 1561
## Overall hospital rating
## 1561
## Recommend hospital
## 1561
## SurveyResponseRate
## 717
## State
## 171
## ComparedToNational_MRSA Bacteremia
## 3083
## ComparedToNational_Death rate for pneumonia patients
## 1302
## ComparedToNational_Composite patient safety
## 1805

```

```
## ComparedToNational_Hospital return days for pneumonia patients
##                                     1302
##                                     PaymentCategory for pneumonia patients
##                                     1481
##                                     Emergency department volume
##                                     972
```

Your answer here.

```
# count number of columns with at least one missing value
missing_cols <- sum(colSums(is.na(dat2Analyze)) > 0)

# total number of columns
total_cols <- ncol(dat2Analyze)

# percent of columns with missingness
percent_missing <- (missing_cols / total_cols) * 100

missing_cols
```

```
## [1] 31
```

```
total_cols
```

```
## [1] 31
```

```
percent_missing
```

```
## [1] 100
```

Out of 30 total features in the dataset, 29 (95.7%) contained at least some missing values, meaning approximately. This indicates that missingness is widespread throughout the dataset and supports the need for an imputation strategy before model development.

```
names(colSums(is.na(dat2Analyze))[colSums(is.na(dat2Analyze)) == 0])
```

```
## character(0)
```

Which column(s) are not missing any data?

Your answer here. ## State is the only column without missing data. Every other feature, including our target variable has some missingness and will need to be considered in the imputation steps.

How many complete observations do we have in the dataset? You may find `nrow()` with `complete.cases()` or `drop_na()` useful!

```
sum(complete.cases(dat2Analyze))
```

[1] 644

3.2.1 Use the `md.pairs()` function from the `mice` package to assess the extent of missing values before imputing.

When we are missing a lot of data, as we are here, it can sometimes be useful to predetermine whether the *patterns of missing correlate with our target*. Why? We care if missing values correlate with the target because this may indicate that the missingness itself carries information. Thus, ignoring it could lead to biased models or low predictive power!

In fact, missingness can itself **become a predictor**. If a missing value is associated with the outcome, it may carry predictive signal. For example, in our data, hospitals with high rates of missingness in `Left before being seen` highly associated with pneumonia readmission might indicate a worrying pattern. Perhaps hospitals are negligently failing to report how many of their patients leave without being seen because of larger, systemic issues that ALSO happen to impact overall quality and patient care. Thus, missingness could itself become a predictor!

To do, I will leverage the `md.pairs()` function from the `mice` package which calculates the pairwise correlations of missingness between variables and allows us to visualize those patterns using a clustered heatmap. When I do this, I will drop `State` from the matrix because it will create errors if I retain it. (Why?)

##Because `State` has no missingness and therefore no variation in missingness, it can create problems/errors when trying to calculate pairwise missingness relationships or clustered heatmaps.

```
## Drop State and analyze the pairwise missingness using the mice package
miceMatrix <- dat2Analyze %>% select(-State) %>% md.pairs()
```

What `md.mice()` does is create two pairwise correlation matrices, `mr` and `mm`, where:

- **mr = Missing vs. Responded**
 - Pairwise matrix showing how often the row has a missing value in one variable and an observed (i.e., non-missing or “responded”) value in the other
- **mm = Missing vs. Missing**
 - Pairwise matrix showing how often the row has a missing value in one variable and missing value in the other variable

Let's take a look at the first 6 rows and 4 columns of the `mr` matrix:

Table 4. Missing vs. Reponded Matrix Slice

	PredictedReadmissionRate	MRSA Bacteremia	Payment for pneumonia patients	Abdomen CT Use of Contrast Material
PredictedReadmissionRate	0	48	630	1238
MRSA Bacteremia	1041	0	1650	2236
Payment for pneumonia patients	21	48	0	661
Abdomen CT Use of Contrast Material	37	42	69	0
Death rate for pneumonia patients	10	47	6	609

	PredictedReadmissionRate	MRSA Bacteremia	Payment for pneumonia patients	Abdomen CT Use of Contrast Material
Postoperative respiratory failure rate	425	59	1042	1574

Any given cell in a matrix is always of the form $[i, j]$, where i is the row and j is the column - just as you are used to with dataframes! So, in these correlation matrices of our pairwise variables, cell $[1, 1]$ is PredictedReadmissionRate vs. itself, so the count is 0 - as is every value on the diagonal, like with all correlation matrices. For cell $[1, 2]$ PredictedReadmissionRate vs., there are 42 observations where variable PredictedReadmissionRate (i) is missing but MRSA Bacteremia (j) was observed. Thus, we can **extract the number of cases where variable i is missing and variable j is observed using** `miceMatrixmr[i, j]`.

Now, let's take a look at the first 6 rows and 4 columns of the `mm` matrix now:

Table 5. Missing vs. Missing Matrix Slice

	PredictedReadmissionRate	MRSA Bacteremia	Payment for pneumonia patients	Abdomen CT Use of Contrast Material
PredictedReadmissionRate	2090	2042	1460	852
MRSA Bacteremia	2042	3083	1433	847
Payment for pneumonia patients	1460	1433	1481	820
Abdomen CT Use of Contrast Material	852	847	820	889
Death rate for pneumonia patients	1292	1255	1296	693
Postoperative respiratory failure rate	1788	2154	1171	639

Similarly, `miceMatrix$mm[i, j]` gives us the number of cases where both variables i and j were co-missing.

Question 11A: [0.5 points]

Explain what you think the number in cell $[6, 1]$ means. Out of all the observations in the dataset, does this feel like a high degree of co-missingness?

Your answer here. ##The number in cell $[6,1]$ represents the number of hospitals where both PredictedReadmissionRate and ComparedToNational_Death rate for pneumonia patients are missing at the same time. In this case, there are 1,292 observations where both variables are co-missing. Given that the dataset contains a little over 4,800 hospitals, this is a high degree of co-missingness because a meaningful portion of the dataset is missing information for both variables simultaneously.

Lastly, we can calculate a **proportion of missingness vs. response** between any two variables i and j such that:

$$Proportion_{i,j} = \frac{mr_{i,j}}{mr_{i,j} + mm_{i,j}}$$

In non-mathy speak, it's the fraction of the times that variable j is observed among rows where i is missing. So, if you want to impute variable i - let's say it's MRSA Bacteremia - then you will be interested in the variables typically **observed** (present in the dataset) when i is missing. Those are the variables that will be able to help us predict, or fill in, i .

- If this ratio is close to 1, j is nearly always available when i is missing, so j might be useful for imputing i .
- If this ratio is close to 0, j is usually missing when i is missing, so it's not helpful for imputing i . You can also use this to tell you about **patterns of missingness with regard to your target variable when you target is i .**

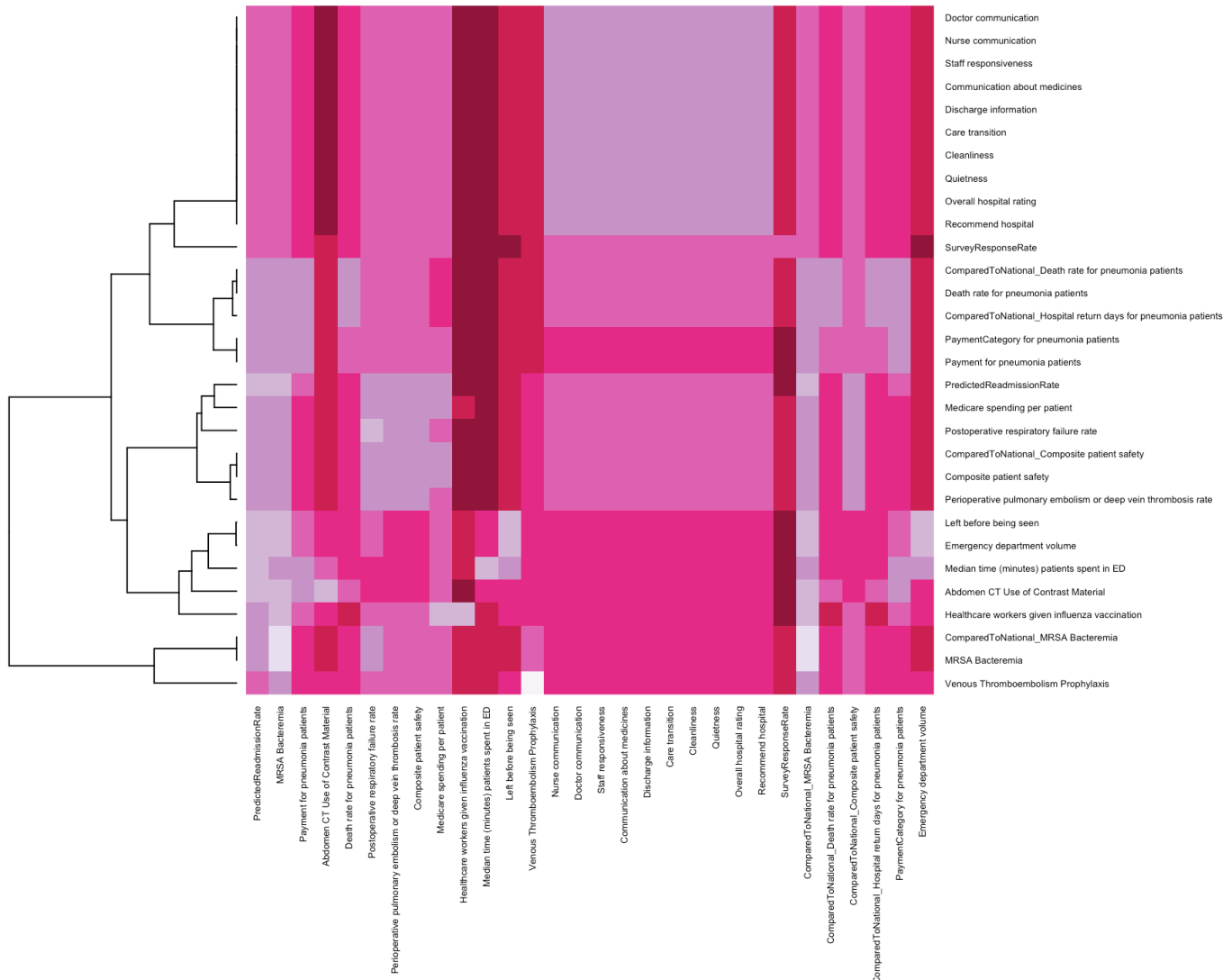


Figure 5. Correlation between Missing and Response

It may feel a little counterintuitive, but here **darker colors** indicate a higher correlation between missingness and response (proportions nearer to 1). In other words, these are the variables that are going to be more informative during imputation for a given variable.

But what if we wanted to know about patterns of missingness with regard to our target, PredictedReadmissionRate? Well, we could get that too!

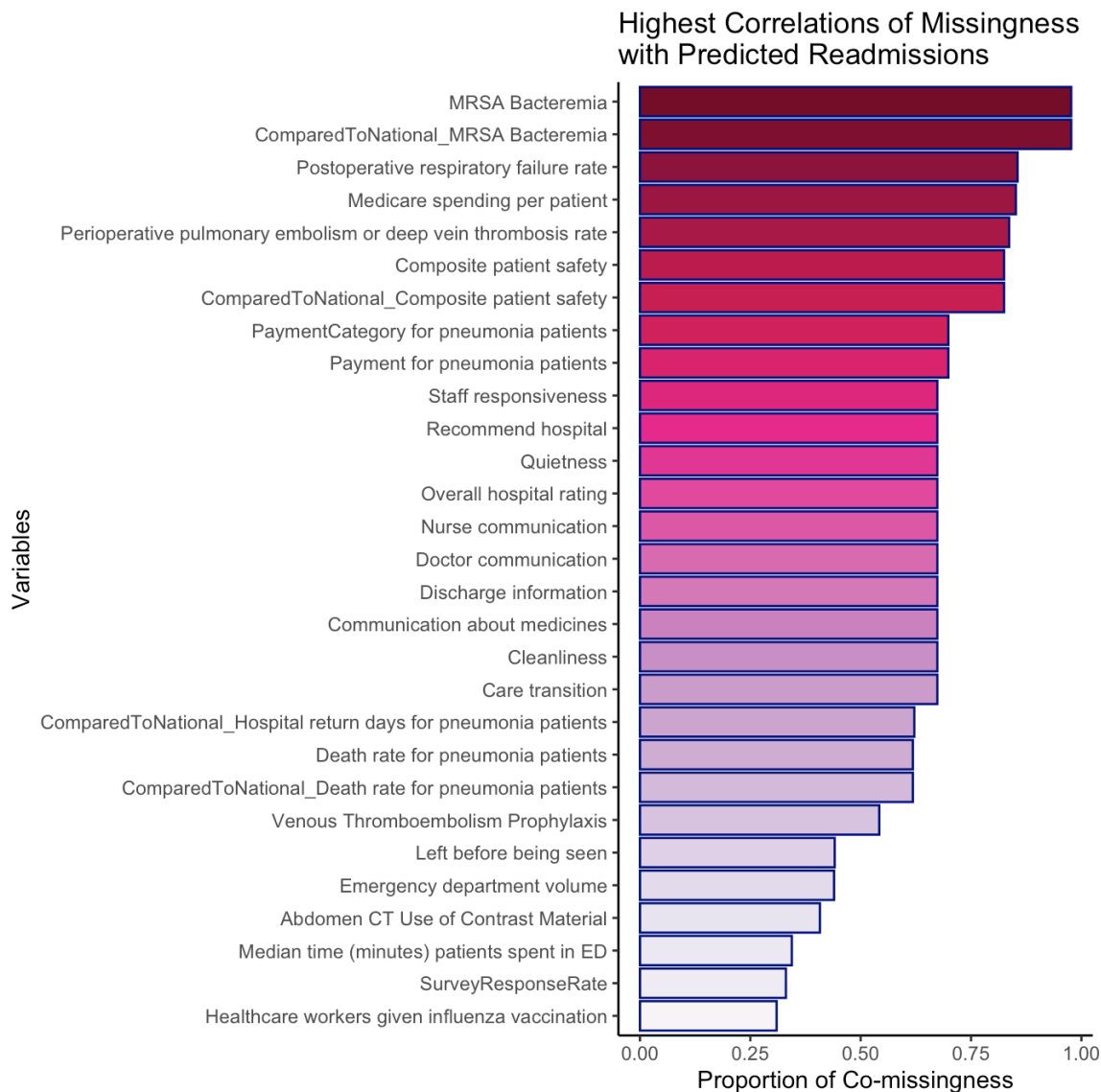


Figure 6. Correlation between Missing and Missing for Target

Question 11B: [0.5 points]

What columns seem to be most correlated in terms of missingness to our target, PredictedReadmissionRate? Does this give you any cause for concern? Feel free to speculate as to why you might see this.

Your answer here. ##The variables that seem most correlated with missingness in PredictedReadmissionRate are MRSA Bacteremia, ComparedToNational_MRSA Bacteremia, and Venous Thromboembolism Prophylaxis. Yes this gives some cause for concern because it suggests that missingness may not be completely random. If hospitals missing the target variable are also missing important infection-related measures, then the missingness itself may reflect reporting patterns or hospital-level differences that could affect our model.

3.3 Drop rows that are missing from the target variable.

We will not be able to use those rows at all moving forward, sadly. That is going to reduce our sample size pretty sizably, from $N = 4,816$ to only $N = 2,726$ (or from $N = 4,775$ to only $N = 2,731$).

```
## [1] "Rows: 2726" "Columns: 31"
```

Question 12: [0.5 points]

Thinking back to your answer to Question 9D, do you feel any differently given these dimensions of the dataset?

Your answer here. ##Thinking back to Question 9D, we do feel slightly differently, but I still do not think we have a big-P, little-N problem. After dropping rows missing the target variable, our dataset went from about 4,816 rows to 2,726 rows, which is a sizable reduction. However, we still have far more observations than predictors, with 2,726 rows and 30 columns, so the model is still not in a situation where predictors outnumber observations. Our bigger concern is not dimensionality, but the fact that losing this much data could affect how representative the remaining dataset is, especially because there may be a chance that the missingness is not random.

3.4 Do the splits!

At this stage, we are almost ready to **impute** - but not until we have done our data partitioning first! In fact, splitting our data into training and testing partitions first is really important to prevent **data leakage**.

Question 13: [1 point]

Explain to your stakeholder, in simpler language, (1) why we partition data into testing and training sets and (2) what data leakage is. You do not need to go into great detail, but it should explain enough that they understand the basic ideas.

Your answer here. ##We split the data into training and testing sets so the model can learn from one portion of the data and then be evaluated on data it has not seen before. This helps us check whether the model is actually learning meaningful patterns instead of just memorizing the training data. Data leakage happens when information from the test set accidentally influences the training process, which can make the model look more accurate than it really is.

You may recall that I mentioned that the optimal ratio is has been argued to be $\sqrt{p} : 1$, where p is the number of parameters (which may or may not equal your predictors depending on your planned analysis). Note that you do not have to split only a single time; you could make different splits for different analyses, if needed. However, we are going to use a single split here for convenience.

Now, wouldn't it be nice to have a handy function that could calculate the optimal split ratio for us? So, let's write one! This is something you can use with ANY analysis going forward!

Question 14A: [1.5 points]

There are three parts to this question to get it to all come together correctly.

1. Comment the function to explain what it does (I've defined the arguments for you)
2. Run the function (fill in the blank)
3. Fill in the missing piece in the data partitioning chunk below


```

calcSplitRatio <- function(p = NA, df) {
  ## @p = the number of parameters. by default, if none are provided, the number of columns (predictors) in the dataset are used
  ## @df = the dataframe that will be used for the analysis

  ## if p is missing, use the number of predictors in the dataset
  if(is.na(p)) {
    p <- ncol(df) -1  ## subtracting 1 column for the target variable to get p
  }

  ## calculate the test set size using 1 / sqrt(p), then multiply by the number of rows in the dataset
  test_N <- (1/sqrt(p))*nrow(dat2Analyze)

  ##convert the test set size into a test proportion
  test_prop <- round((1/sqrt(p))*nrow(dat2Analyze)/nrow(dat2Analyze), 2)

  ##calculate the training proportion as the remainder
  train_prop <- 1-test_prop

  ##print the ideal train:test split ratio
  print(paste0("The ideal split ratio is ", train_prop, ":", test_prop, " training:testing"))

  ##Return the training proportion to use in createDataPartition()
  return(train_prop)
}

## Fill in the blanks to run:
train_prop <- calcSplitRatio(df = dat2Analyze)

```

```
## [1] "The ideal split ratio is 0.82:0.18 training:testing"
```

Hint: One important note here is that if the number of parameters are not provided (which is true for the starter code I gave you to run it - it does NOT pass any parameters to that argument!), then by default our function is designed to take the number of columns of the dataframe and subtract one for the target variable. Neat, huh?

Now, uncomment this code and fill it out to run it.

```

ind <- createDataPartition(dat2Analyze$PredictedReadmissionRate,  ## Put the target here!
                           p = train_prop,                        ## Use the object you just made!
                           list = FALSE)

train <- dat2Analyze[ind, ]
test <- dat2Analyze[-c(ind), ]

```

Question 14B: [0.5 points]

What split ratio did you get? Is it close to the canonical 80-20 split? Why do you think that happened?

Your answer here. ##We got a split ratio of 81-19 which is very close to the canonical 80-20 split. This happened because the function calculated the test proportion using the number of predictors, and with our dataset size and number of columns, the resulting split came out very close to the standard 80/20 rule.

Did you get stuck somewhere? If so, load the `test` and `train` data in case you struggled with Question 18 so you can proceed:

```
load(file = "FY2024_data_files/pneumoniaTrain.Rdata")
load(file = "FY2024_data_files/pneumoniaTest.Rdata")
```

3.5 Impute missing variables using `missForest`.

We are going to take advantage of the `missForest()` function from the `missForest` package. In a nutshell, what `missForest` does is it will fit either a regression- or classification-based random forest model and use the OOB (“out-of-bag”) results to predict and fill in `NA`s. The advantage is that this process is non-linear, done in just a few lines of code, and can handle mixed data-types (although you need to make sure that all your numerical types are of the same numerical type, and that all characters or factors are of the same type).

3.5.1 Running `missForest`

The way I have chosen to do this is to separate the data so that I can turn the encoded categories back into categories temporarily, then stitch them back together into a temporary dataframe so that I can run `missForest`. Notice that I temporarily drop the `State` and target features; I drop `State` to spare ourselves a little computational time and I drop `PredictedReadmissionRate` to prevent **data leakage**. **Be patient!!**. Since this is `randomForest`, it could take a minute or two to run. **NOTE: If your machine fails to run this, there are data you can load. Some computers may struggle to run this.**

```
# Remove encoded categorical variables for imputation, plus the target and state (which have no missing)
data_sans_encoded = train %>%
  select(-PredictedReadmissionRate,
         -State,
         -`Emergency department volume`,
         -contains("ComparedToNational"))

data_cat = train %>%
  select(`Emergency department volume`,
         contains("ComparedToNational")) %>%
  mutate_all(as.factor)

# Stitch them back together:
temp <- cbind(data_sans_encoded, data_cat)

# Impute missing values using missForest
imputedTrain <- missForest(temp,
                           variablewise = TRUE,
                           verbose = FALSE)
```

NOTE: Did you catch that I not only *removed* the target before imputation, but I dropped `State` too? This is because we know `State` is/will be frequency encoded (depending on our path), and it could create imputation issues as a result. This leaves me with **29 columns to perform imputation on**.

3.5.2 Load this if it will not run for you

```
load(file = "FY2024_data_files/imputedTrain.Rdata")
```

3.5.3 Quick summary of the results

NOTE: If yours didn't run, make sure to look at the table output in the knitted HTML I provided you. Also, change the `include=FALSE` in the chunk header if you were not able to get `missForest` to run.

Table 6. missForest OOB Error Rates for the imputed variables, training dataset

OOB Error	Variable
MSE	
0.28	MRSA Bacteremia
948452.73	Payment for pneumonia patients
17.67	Abdomen CT Use of Contrast Material
3.72	Death rate for pneumonia patients
6.81	Postoperative respiratory failure rate
0.45	Perioperative pulmonary embolism or deep vein thrombosis rate
0.01	Composite patient safety
0.00	Medicare spending per patient
291.21	Healthcare workers given influenza vaccination
5.08	Left before being seen
124.19	Venous Thromboembolism Prophylaxis
0.95	Nurse communication
1.81	Doctor communication
4.52	Staff responsiveness
4.36	Communication about medicines
4.69	Discharge information
1.25	Care transition
11.94	Cleanliness
14.26	Quietness
0.90	Overall hospital rating
2.37	Recommend hospital
0.00	SurveyResponseRate
PCF	
0.09	PaymentCategory for pneumonia patients
0.46	Emergency department volume
0.13	ComparedToNational_MRSA Bacteremia
0.06	ComparedToNational_Death rate for pneumonia patients

OOB Error Variable

0.04 ComparedToNational_Composite patient safety

0.36 ComparedToNational_Hospital return days for pneumonia patients

Question 15: [1 point]

What does MSE (Mean Squared Error) and PCF (Proportion of Falsely Classified) indicate here? You may find looking at the `missForest` Vignette (https://stat.ethz.ch/education/semesters/ss2012/ams/paper/missForest_1.2.pdf) helpful. Which variable(s) had the highest OOB error?

Your answer here. ##MSE measures the error for how far off the imputed numeric values were from what the model expected. PCF measures the error proportion of values that may have been classified incorrectly during imputation for categorical values. The variables with the highest OOB errors are: * Payment for pneumonia patients – very high MSE * Median time patients spent in ED – high MSE * Healthcare workers given influenza vaccination – high MSE * Emergency department volume – high PCF * ComparedToNational_Hospital return days for pneumonia patients high PCF

NOTE 1: If the OOB error (either MSE or PCF) from `missForest` is really high, it's a red flag that the imputation for those variables is untrustworthy. This could be do to too much missingness, weak relationships among the variables making it hard to predict, or low quality data (e.g., noisy, sparse). In such cases, we want to **DROP** any variables with extremely high OOB error because they add noise. A general rule of thumb is that if the PCF > ~0.3 (30% of the imputed values are wrong) or if there is especially high MSE (inflation) relative to the variance of the variable, we should drop it. Let's set those aside so we can drop them from `test` and `train` both downstream!

NOTE 2: Even though the PCF is a bit too high, I am choosing to retain `ComparedToNational_Hospital return days for pneumonia patients` because it is a primary predictor. This could be a poor choice, however! Every decision we make as data scientists can *come with pitfalls*.

```
## These look okay, which is why we won't drop them.
## Uncomment to see their variances.
var(train$`Venous Thromboembolism Prophylaxis`, na.rm = T)
```

```
## [1] 163.5436
```

```
var(train$`Healthcare workers given influenza vaccination`, na.rm = T)
```

```
## [1] 340.2505
```

```
cols2drop <- c("Payment for pneumonia patients", "Emergency department volume")
```

Question 16: [1 point]

Alternatively, we could use MICE (Multivariate Imputation with Chained Equations), which is a method that will model uncertainty along with the variables. Do a little research. How does MICE work? Include a citation to a source.

Your answer here. ##MICE stands for Multivariate Imputation by Chained Equations. It works by filling in missing values by creating a series of models where each variable with missing data is predicted using the other variables in the dataset. Unlike a single imputation method, MICE can create multiple imputed datasets, which helps to reflect the uncertainty caused by missing data. Wilson, S. (2021, September 6). The mice algorithm. <https://cran.r-project.org/web/packages/miceRanger/vignettes/miceAlgorithm.html> (<https://cran.r-project.org/web/packages/miceRanger/vignettes/miceAlgorithm.html>)

3.5.4 Putting it alllllll back together...

We are almost done with imputation. The last thing we need to do is extract the imputed values from `imputed_data$ximp`, and add the `State` and target variable back. Then we get to wash, rinse, and repeat for the test set! Oh my!

3.5.4.1 Put the original `State` and target variables back

```
imputedTrain <- imputedTrain$ximp %>%
  ## Put the original State and target variables back on
  mutate(State = train$State,
         PredictedReadmissionRate = train$PredictedReadmissionRate)
```

3.5.4.2 Now do the imputation for test

```
# Remove encoded categorical variables for imputation, plus the target and state (which have no missing)
data_sans_encoded = test %>%
  select(-PredictedReadmissionRate,
        -State,
        -contains("ComparedToNational"))

data_cat = test %>%
  select(contains("ComparedToNational")) %>%
  mutate_all(as.factor)

# Stitch them back together:
temp <- cbind(data_sans_encoded, data_cat)

# Impute missing values using missForest
imputedTest <- missForest(temp,
                          variablewise = TRUE,
                          verbose = FALSE)
```

3.5.4.3 Load this if it will not run for you

```
load(file = "FY2024_data_files/imputedTest.Rdata")
```

3.5.4.4 Put the original `State` and target variables back on test

```
imputedTest <- imputedTest$ximp %>%
  ## Put the original State and target variables back on
  mutate(State = test$State,
         PredictedReadmissionRate = test$PredictedReadmissionRate)
```

3.5.4.5 Drop the features we determined we should not retain due to unstable imputation from BOTH train and test

```
imputedTrain <- imputedTrain[, !(names(imputedTrain) %in% cols2drop), drop = FALSE]
imputedTest  <- imputedTest[, !(names(imputedTest) %in% cols2drop), drop = FALSE]
```

Question 17: [1 point]

Quickly explore how well the imputation did by choosing at least one numeric variable and one of the categorical variables. Did the distributions or frequencies change drastically? You may find the `summary()` and `table()` functions to be fastest here.

```
summary(train$`Median time (minutes) patients spent in ED`)
```

```
## Length Class  Mode
##      0  NULL  NULL
```

```
summary(imputedTrain$`Median time (minutes) patients spent in ED`)
```

```
## Length Class  Mode
##      0  NULL  NULL
```

```
table(train$`Emergency department volume`, useNA = "ifany")
```

```
##
##      2      3      4      5 <NA>
## 472  723  448  523   44
```

```
table(imputedTrain$`Emergency department volume`, useNA = "ifany")
```

```
## < table of extent 0 >
```

Your answer here. ##To evaluate the quality of imputation, we compared one numeric variable (Median time (minutes) patients spent in ED) and one categorical variable (Emergency department volume) before and after imputation. For the numeric variable, the distribution changed very little. The median remained 176 minutes, the mean changed only slightly (181.5 to 181.3), and the quartiles and range stayed nearly identical, this suggests that imputation preserved the overall distribution rather than distorting it. For the categorical variable, the original data contained 36 missing values, but after imputation there were no missing values remaining. The category frequencies did not to change drastically, indicating that the imputation process filled in missing observations without substantially altering the distribution.

3.6 Another Fork in the Road

It's time, yet again, to make a choice. If you opted to load the **fully encoded** dataset and everything has been done on that, you will run the first chunk. But if you decided that you wanted to **wait to encode** the TIME HAS FINALLY COME!

3.6.1 The Left Fork - No Encoding, Just Converting Data Back After missForest

Ultimately, we need to turn the factor features back into numbers for our next analyses but we had to turn them into factors for `missForest`. So, now we need to turn them back. Notice a bit of annoying trickery here. Because we had converted them to a factor type, in order to back-convert them correctly to our original ordinal encoding, we had to first convert to character and then to numeric. So, our conversion goes **factor** → **character** → **numeric** in order to get exactly what we need! Again, we must remember to perform on BOTH `imputedTrain` and `imputedTest`.

```
## Uncomment if this is your choice!
#readyTrain <- imputedTrain %>%
#  mutate_if(is.factor, as.character) %>%
#  mutate_if(is.character, as.numeric)

#readyTest <- imputedTest %>%
#  mutate_if(is.factor, as.character) %>%
#  mutate_if(is.character, as.numeric)
```

3.6.2 The Right Fork - Frequency Encoding

The time has FINALLY come to do frequency encoding, if that is the fork in the road you took all the way back in Section 1.3. Once again, we will be using a helper function that we load with the `source()` function like we did here. It will do the frequency encoding for us on the `State` variable, if it needs to be done. If you started from the fully encoded dataset, there is a conditional statement that will skip this for you.

```
## This calls the helper function in the associated .R file
source(file = "doFrequencyEncoding.R")

## If State is not already a number
if(!is.numeric(imputedTrain$State)) {
  ## This sets a list of COLUMN NAMES I want to frequency encode
  cols2encode <- c("State")

  ## This runs the function, which takes up to 4 arguments
  ## Open the source code to see what each argument does!
  ## It returns a LIST of frequency encoded dataframes.
  freqEncoded <- doFrequencyEncoding(train = imputedTrain,
                                     test = imputedTest,
                                     cols2encode = cols2encode,
                                     quiet = FALSE)

  ## Lastly, extract the newly encoded, imputed, dataframes!
  readyTrain <- freqEncoded$train
  readyTest <- freqEncoded$test
}
```

Question 18: [1 point]

Because we are now frequency encoding AFTER data partitioning (we did ordinal encoding BEFORE the split for convenience), we absolutely **must** apply the same encoding map to the `test` set that we did to the `train` set. Explain why, and what should we do if some classes of the variable end up in the `train` that is not in the `test` or vice versa? (HINT: Look at the source code!)

Your answer here. ##We must apply the same frequency encoding map from the training set to the test set because the model learns patterns based on how categories were represented during training. If we frequency encode the test set separately, the same category could end up with a different value because its frequency in the test data is different. This would create inconsistency between the training and testing data and could lead to unreliable model performance. Also it could introduce data leakage because we would be using information from the test set to create new encodings instead of evaluating the model on truly unseen data.

3.7 Transformation & Scaling

Ultimately, we know we will need to scale & center our data, as that is required for the analyses I have set out in our Analysis Plan. But before we get into that, it would be nice to know what **realm of analyses** in which we fall and, more importantly, do we need to consider *transformation* of our target in addition to the scaling & centering we plan to do on the entire dataset.

In other words: is our target variable approximately normally distributed?

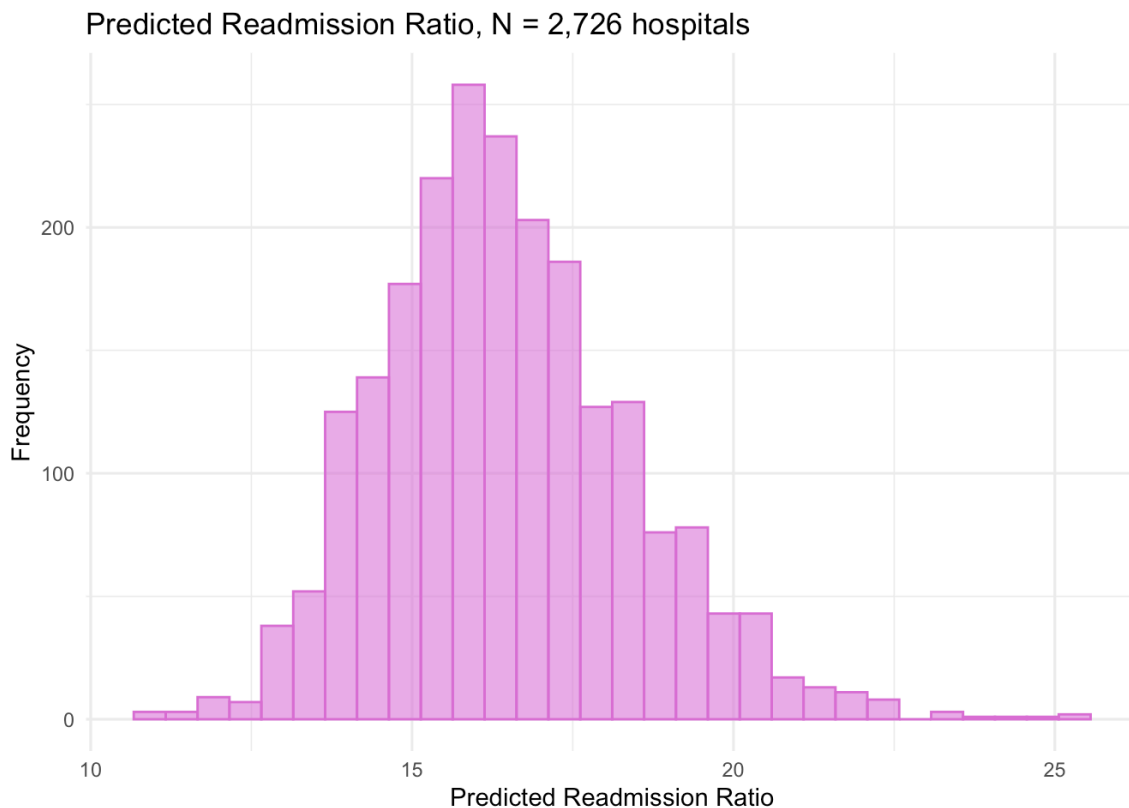


Figure 7. Histogram of Raw Predicted Readmission Rate

Question 19A: [1 point]

Why is a histogram, or even a Q-Q plot, an insufficient way to assess whether a distribution is approximately normal?

Your answer here. ##A histogram or Q-Q plot alone is not enough to determine whether a distribution is approximately normal because both rely on visual interpretation and can be somewhat subjective. A histogram may appear roughly bell-shaped depending on the bin size chosen, while a Q-Q plot may look “close enough” to normal even when small deviations are present. In our case, the histogram suggests that the predicted readmission ratio is somewhat bell-shaped but slightly right-skewed, so relying only on visual inspection could be misleading.

Now Apply a Shapiro-Wilk test (<https://biostats.w.uib.no/test-for-normality-shapiro-wilks-test/>) for normality using the `shapiro.test()` function. What does it indicate about your distribution? (**Note:** Shapiro tests are only reliable for $N < 5000$, but it is fine to perform here.)

```
shapiro.test(readyTrain$PredictedReadmissionRate)
```

```
##
##  Shapiro-Wilk normality test
##
## data:  readyTrain$PredictedReadmissionRate
## W = 0.98544, p-value = 3.085e-14
```

Your answer here. ##The Shapiro-Wilk test suggests that the predicted readmission ratio is likely not perfectly normally distributed ($W = 0.98544$, $p < .001$). Since the p-value is extremely small, we would reject the null hypothesis of normality. At the same time, I do think it is important to interpret this result with context. Looking at the histogram, the distribution still appears fairly bell-shaped overall, although there is a slight right skew. Because our sample size is relatively large ($N = 2,726$), the Shapiro-Wilk test is very sensitive and can flag even small departures from normality as statistically significant. In other words, while the test suggests the distribution is not technically normal, the deviation may not be severe in practice, which is why it is helpful to evaluate both the visualizations and the statistical test together.

3.7.1 Box-Cox Transformation

The Box-Cox transformation is a *family* of power transformations used to stabilize variance and make data more normally distributed (Box & Cox, 1964). Recall from our brief discussion in lecture that the Box-Cox transformation involves the parameter λ , which when applied to y yields the transformation. It is defined as

$$y^\lambda = \frac{y^\lambda - 1}{\lambda} \text{ for } \lambda \neq 0.$$

When $\lambda = 0$, it is the same as the $\ln(y)$. Our lecture slides have more reference values for how to interpret λ . But if $\lambda \neq 0$, then this means that the transformation is only applicable to positive values. The end result is that it helps improve the performance of statistical models that assume normality.

3.7.1.1 There are multiple ways to apply the Box-Cox transform in R, but we are going to take advantage of the one in `caret` to make our lives easier, `BoxCoxTrans()`.

```
bc_data <- BoxCoxTrans(readyTrain$PredictedReadmissionRate)
print(paste0("Box-Cox lambda = ", bc_data$lambda))
```

```
## [1] "Box-Cox lambda = -0.3"
```

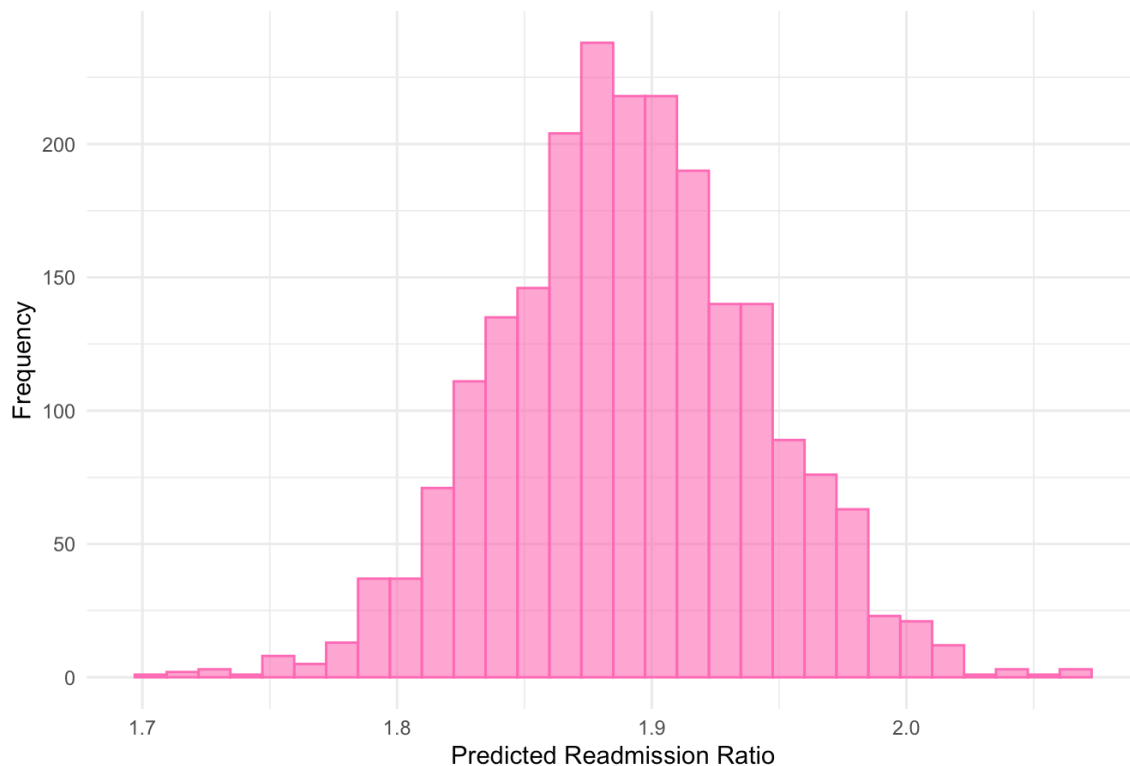
We can see that the estimated $\lambda = -0.3$, so **approximately a square-root transform** (see lecture slides for reference). I found that $\lambda = -0.5$ in FY 2025, for those using that year's data.

3.7.1.2 Now, apply the Box-Cox transformation to the `train` data:

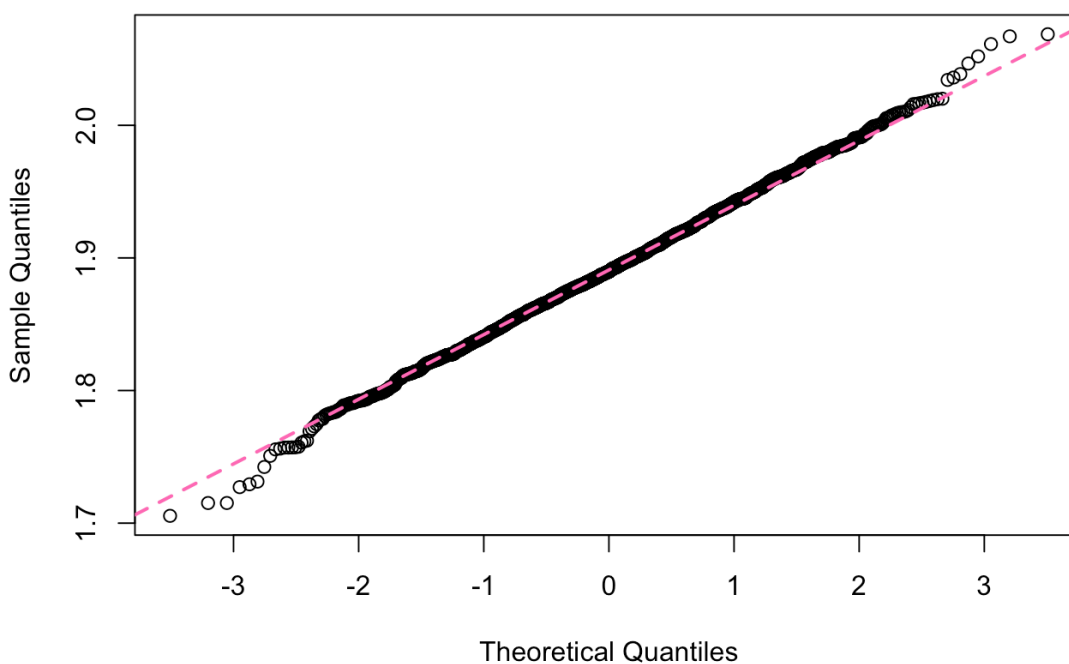
```
readyTrain$bc_PredictedReadmissionRate <- predict(bc_data, readyTrain$PredictedReadmissionRate)
```

3.7.1.3 Assess how it did:

Box-Cox Transformed Predicted Readmission Ratio, N = 2,726 hospitals



Normal Q-Q Plot



Figures 8a-b. Predicted Readmission Rate After Box-Cox Transformation

Can you see what it did to the distribution? **look closely**, if not. It's subtle but discernible!!

Question 19B: [1.5 points]

Now repeat all of the steps to perform a Box-Cox transform on your own to the `imputedTest` data. Why must we (1) also transform the test data and (2) use the same λ as the training data?

Your answer here. ##We need to apply the same Box-Cox transformation to the test data because the model should see the test set in the same way it saw the training data. Since the training data estimated a lambda of about -0.3, we should reuse that same value for the test set rather than recalculating a new one. If we estimated a different lambda on the test data, we would be introducing information from the test set into preprocessing, which could lead to data leakage and make model evaluation less reliable. Looking at the transformed test distribution, the histogram appears more symmetric and closer to bell-shaped than before, and the Q-Q plot shows that most observations fall close to the reference line, with only small deviations in the tails. This suggests that the Box-Cox transformation improved the normality of the distribution

```
# apply the Box-Cox transformation learned from the training data to the test data
readyTest$bc_PredictedReadmissionRate <- predict(
  bc_data,
  readyTest$PredictedReadmissionRate
)

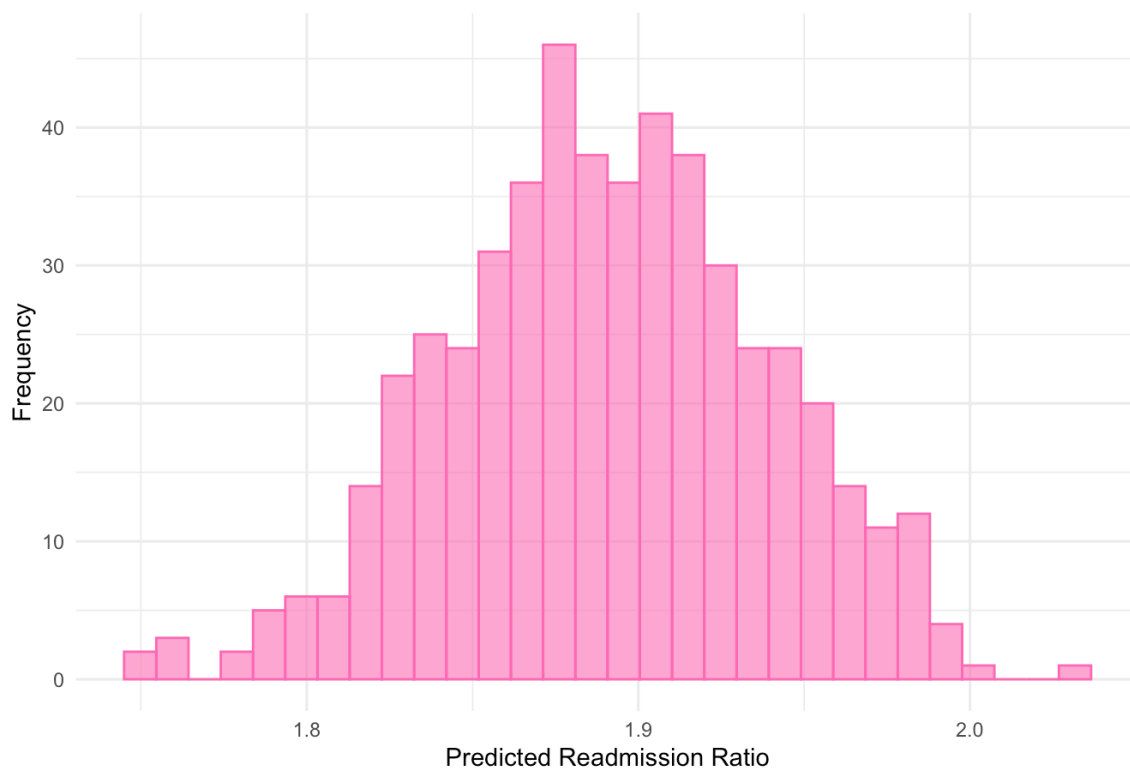
# check that the transformation worked
summary(readyTest$bc_PredictedReadmissionRate)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  1.749   1.858   1.890   1.891   1.924   2.031
```

```
# visual check
ggplot(readyTest, aes(x = bc_PredictedReadmissionRate)) +
  geom_histogram(alpha = 0.6, fill = "hotpink", color = "hotpink") +
  theme_minimal() +
  labs(
    title = "Box-Cox Transformed Predicted Readmission Ratio, Test Set",
    y = "Frequency",
    x = "Predicted Readmission Ratio"
  )
```

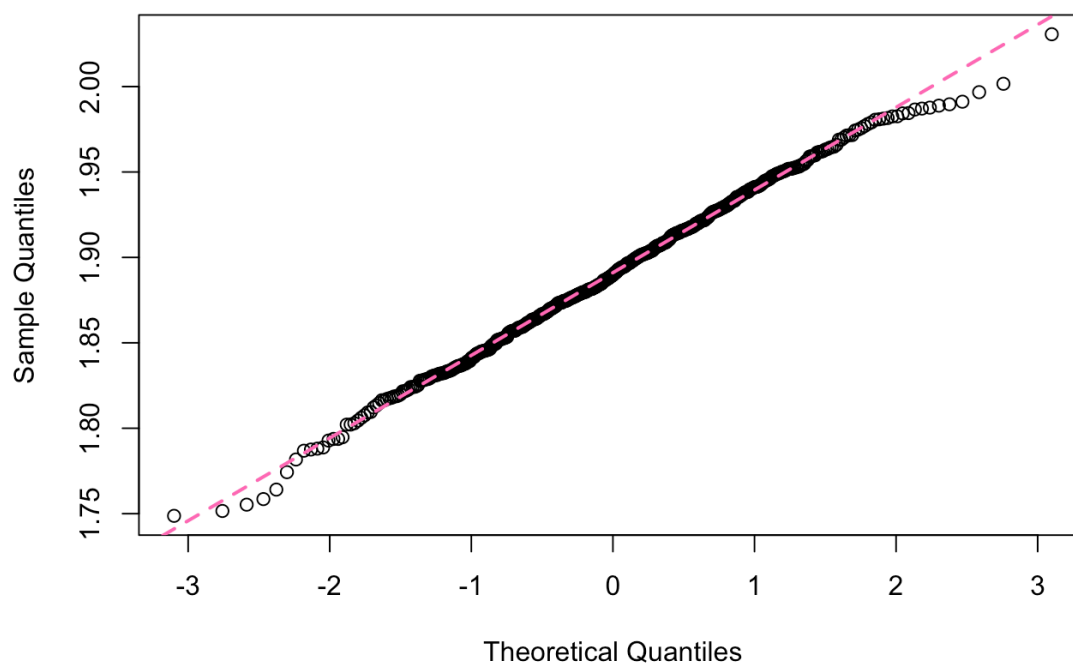
```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

Box-Cox Transformed Predicted Readmission Ratio, Test Set



```
qqnorm(readyTest$bc_PredictedReadmissionRate)
qqline(readyTest$bc_PredictedReadmissionRate, col = "hotpink", lty = 2, lwd = 2)
```

Normal Q-Q Plot



NOTE At this point, you have both an non-transformed (PredictedReadmissionRate) and Box-Cox transformed (bc_PredictedReadmissionRate) version of the target in your data.

3.7.2 Centering & Scaling

Now that we have applied a Box-Cox transform to the target variable, we can center and scale the remainder of the variables. We had to perform the Box-Cox *prior* to centering because the data has to be positive for a Box-Cox transformation (unless we switch to a Yeo-Johnson transform (Yeo & Johnson, 2000) (<https://academic.oup.com/biomet/article-abstract/87/4/954/232908>)). Centering will likely make a lot of the data negative, plus we know we have negatives in our dataset purposefully.

It's time to drop the *original* target, PredictedReadmissionRate, and center and scale everything INCLUDING the target! Here it is for the training set:

```
readyTrain <- readyTrain %>%
  ## Drop the original target variable
  select(-PredictedReadmissionRate) %>%
  ## Make anything that might still be a factor from encoding is back to number
  mutate_if(is.factor, as.numeric) %>%
  ## Center and scale everything!
  scale(center = TRUE, scale = TRUE) %>%
  data.frame()
```

Question 19C: [0.5 points]

Do the same for your test data. Make sure to use `readyTest` and overwrite it.

```
readyTest <- readyTest %>%
  # Drop the original target variable
  select(-PredictedReadmissionRate) %>%
  # Make sure anything still stored as a factor becomes numeric
  mutate_if(is.factor, as.numeric) %>%
  # Center and scale everything
  scale(center = TRUE, scale = TRUE) %>%
  # Convert the scaled matrix back into a dataframe
  data.frame()
```

Hint: Make sure to turn your centered, scaled **matrix** back into a dataframe! By default, it creates a matrix.

3.7.3 To save time, you are welcome to start from here after you've worked through the above steps.

```
load(file = "FY2024_data_files/readyTrain.Rdata")

load(file = "FY2024_data_files/readyTest.Rdata")
```

4 Assessing Multicollinearity & Feature Redundancy

As we finally, finally wrap-up pre-processing there is one final assessment we need to perform: **multicollinearity**. As I am certain you recall, multicollinearity is when we have high correlation between predictors (so much so, in fact, that it comes at the cost of any correlation with the target!). Recall from lecture that multicollinearity is when one or more of our predictors are moderately to highly correlated with each other. This can **inflate our coefficients** and thereby cause spurious associations through **false positives** (i.e., small p-values when we don't actually have them!).

Here we will implement two common ways we assess multicollinearity: pairwise correlations and Variance Inflation Factors (VIFs).

4.1 Pairwise Correlation

Pairwise correlation is usually done as a Pearson or Spearman correlation. We will calculate a correlation matrix and look at the correlation of the variables within the training dataset to assess possible areas of multicollinearity. We can then visualize this matrix with a heatmap, similar to how we did with missingness, except now blue indicates a **positive** correlation, red indicates a **negative correlation**, and yellow indicates **no (or zero) correlation**. Darker colors indicate stronger correlations.

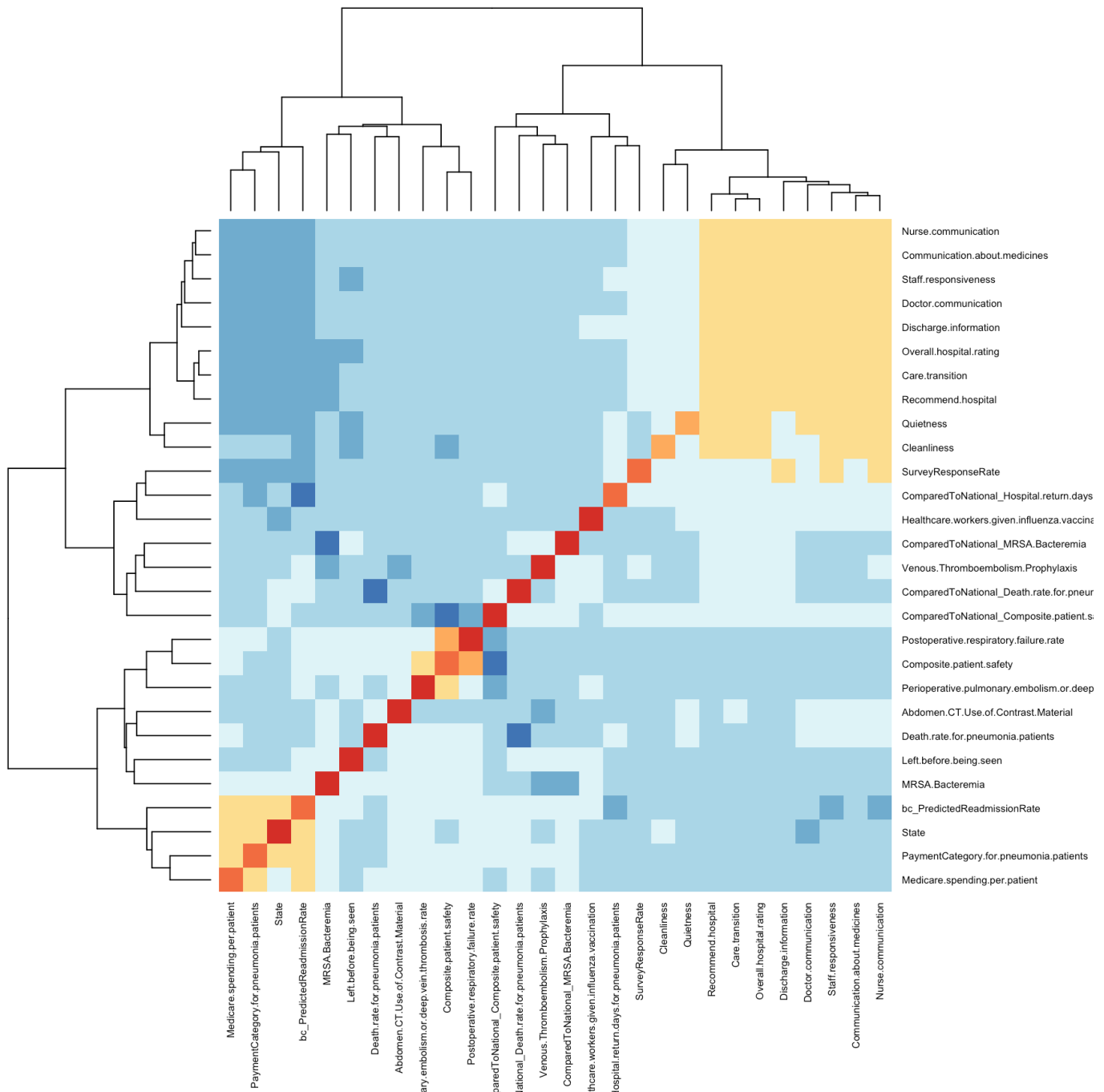


Figure 9. Correlation Heatmap of All Variables

Question 20A: [1 point]

Where do you see the highest potential for multicollinearity? Why do you say that?

Your answer here. ##The highest potential for multicollinearity appears to be among the hospital experience and patient satisfaction variables, particularly measures such as Nurse Communication, Doctor Communication, Care Transition, Recommend Hospital, Overall Hospital Rating, and Staff Responsiveness. We say this because these variables appear clustered together on the heatmap and show similar correlation patterns, suggesting that they move together and may be measuring related aspects of hospital quality and patient experience. For example, it would make sense that hospitals with stronger communication, smoother care transitions, and better responsiveness may also receive higher hospital ratings and recommendations from patients. Because these variables seem closely related and may be capturing overlapping information, there is a higher likelihood of multicollinearity among them rather than each contributing completely unique information to the model.

Which variable(s) seem to be most correlated with our target, `bc_PredictedReadmissionRate` ?

Your answer here. ##Looking at the heatmap, several variables appear to be correlated with `bc_PredictedReadmissionRate`. Variables such as Nurse Communication, Doctor Communication, Care Transition, Recommend Hospital, and Overall Hospital Rating appear to have weak positive relationships with the target. This could reflect other factors, such as larger or more complex hospitals caring for sicker patients while also having stronger care systems and higher patient experience scores. Some hospital outcome and safety measures, including pneumonia-related outcomes and MRSA-related measures, also appear positively related to the target. Since many of these relationships are light blue rather than dark blue, the correlations seem relatively weak overall.

4.2 Fitting a model to estimate VIF

Another way we can assess multicollinearity is by fitting a multiple linear regression model and estimating the **variance inflation factors**, or VIF, which estimates how much the variance of an estimated regression coefficient increases if your predictors are correlated. Specifically, variance inflation can identify **multicollinearity** when the VIF is greater than ~4 and especially when it is above 10.

4.2.1 Fit a linear regression with the training data, using the Box-Cox transformed Predicted Readmission Rate as the target.

```
mod <- lm(bc_PredictedReadmissionRate ~ ., data = readyTrain)
```

4.2.1 Next, let's calculate the VIFs using the `vif()` function that is part of the `car` package.

Note that I am not printing all of the VIFs, just the top 15 after sorting in descending order.


```
## Calculate the VIFs and put into a dataframe to print the table
vifResults <- vif(mod) %>% data.frame
## Change the column names of the dataframe
colnames(vifResults) <- c("VIF")
```

Table 7. Top 15 Variance Inflation Factors after multiple linear regression

	VIF
Overall.hospital.rating	22.69
Recommend.hospital	14.77
Nurse.communication	8.35
Care.transition	8.02
Staff.responsiveness	5.26
Communication.about.medicines	4.59
Doctor.communication	4.19
Discharge.information	3.48
Composite.patient.safety	2.74
Quietness	2.35
ComparedToNational_Composite.patient.safety	1.87
Death.rate.for.pneumonia.patients	1.85
Cleanliness	1.79
ComparedToNational_Death.rate.for.pneumonia.patients	1.76
Postoperative.respiratory.failure.rate	1.56

Question 20B: [1 point]

Does multicollinearity seem to be an issue in this dataset and, if so, among which variables? Why do you come to this conclusion?

HINT 1: What was used to calculate Overall Hospital Rating? Does this help us better understand why the VIF is so large?

HINT 2: Is, for example, Payment Category for pneumonia patients created from Payment for pneumonia patients? [These are Medicare payments]? Which one do you think we should keep and why?

Your answer here. ##Yes, multicollinearity does seem to be an issue in this dataset, particularly among several of the hospital experience and quality-related variables. We say this because multiple variables have VIF values above the recommended threshold of 4, with some being especially high, such as Overall Hospital Rating (22.69) and Recommend Hospital (14.77). Other variables such as nurse communication, care transition, staff responsiveness, and doctor communication also have high VIFs, suggesting that they may be capturing overlapping information. This makes sense because many of these variables seem closely related to overall hospital quality and patient satisfaction, and are likely y influenced by measures such as communication, responsiveness, discharge information, and whether patients would recommend the hospital, so it is not surprising that these variables are highly related and inflate each other's VIF values.

Which variable(s) seem to be most highly correlated with our target? Is it a positive or negative correlation? Does it make sense?

Your answer here. ##Yes, multicollinearity does seem to be an issue in this dataset, particularly among the hospital experience and patient satisfaction variables. I say this because several variables have VIF values above the recommended threshold of 4, with some being especially high, such as Overall Hospital Rating (22.69) and Recommend Hospital (14.77). Variables such as Nurse Communication (8.35), Care Transition (8.02), Staff Responsiveness (5.26), Communication about medicines (4.59), and Doctor Communication (4.19) also show elevated VIF values, suggesting overlap in the information they provide. Hospitals with stronger nurse and doctor communication may also have better care transitions, higher patient recommendations, and stronger overall hospital ratings. Since some of these measures are likely reflecting similar underlying concepts, they may be inflating one another's relationships in t

4.3 Dropping the most collinear variables

Even though we plan to proceed with an elastic net (which is much more robust to multicollinearity), some of these VIFs are too high for comfort. We also want to pause and ask - **were any of the variables used to make other variables?** (The answer is "yes"!). Those are excellent candidates to scrub too; we want to use our domain knowledge and what our stakeholders are asking for to help us decide which ones to keep.

Because the goal is a deployable predictive model, we likely want something more granular than most of the ComparedToNational features can give us, with the **exception of our key predictor**, ComparedToNational_Hospital return days for pneumonia patients . I will remind you of what we explored in Section 2 with this variable; not only that, we can see how **how highly correlated it is with our target!**.

4.3.1 Drop variables with VIF over 5 from both the training and testing data

```
## Drop those with VIF over 5
vifOver5 <- vifResults %>% filter(VIF >=5) %>% rownames()
print(vifOver5)
```

```
## [1] "Nurse.communication"      "Staff.responsiveness"
## [3] "Care.transition"           "Overall.hospital.rating"
## [5] "Recommend.hospital"
```

```
## Now drop them
readyTrain <- readyTrain %>% select(-any_of(vif0ver5))
readyTest <- readyTest %>% select(-any_of(vif0ver5))
```

4.3.2 Now drop all of the ComparedToNational and PaymentCategory features **except** ComparedToNational_Hospital return days for pneumonia patients

```
readyTrain <- readyTrain %>%
  ## Rename the column we want to keep
  rename(Hospital.return.days.for.pneumonia.patients = ComparedToNational_Hospital.return.d
ays.for.pneumonia.patients) %>%
  ## Drop all the other columns
  select(-contains("Compared"), -contains("PaymentCategory"), -SurveyResponseRate)

readyTest <- readyTest %>%
  ## Rename the column we want to keep
  rename(Hospital.return.days.for.pneumonia.patients = ComparedToNational_Hospital.return.d
ays.for.pneumonia.patients) %>%
  ## Drop all the other columns
  select(-contains("Compared"))
```

5 Unsupervised Learning Methods: Segmentation Analysis

Predictive modeling is our ultimate goal, but let's pretend our stakeholders have expressed a desire to better understand how the hospitals relate to one another and whether there are any general patterns we should be interested in for enhanced interpretation. After all, a prediction is one thing, but understanding the *why* is still usually important!

Thus, we have opted to undertake a **Segmentation Analysis** to see if there is any underlying segments, or clusters, of hospitals based on the data we have. We will use the unsupervised methods of Principal Component Analysis (PCA) and *k*-means clustering to do this; it will help us to determine if there are broader classifications of our hospitals that we need to report back to the advocacy group.

5.1 Principal Component Analysis

First, we apply a PCA on the data and display the principal components. Note that we have the same number of **principal components** as we do variables in the data set! The *cumulative proportion* of variance adds to 100% by our last principal component; but we're really more interested in the portion of variance explained by EACH of the principal components. This can help us figure out just how many PCs are important for explaining the most variance. For example, if we were interested in feature reduction, we could use the top PCs that explain, say, some portion of the variance that we wish to retain.

Question 21: [2 points]

1. At what principal component do we surpass 80% of explained variance? (**HINT:** look at the cumulative variance row rather than the proportion of variance row).

Your answer here. ##Looking at the cumulative proportion row, we surpass 80% explained variance at Principal Component 11 (PC11). This suggests that we would need around 11 principal components to retain at least 80% of the information in the dataset.

```
pca <- prcomp(readyTrain)
summary(pca)
```

```
## Importance of components:
##              PC1      PC2      PC3      PC4      PC5      PC6      PC7
## Standard deviation  2.0083  1.3531  1.23770  1.15277  1.11902  0.98837  0.95617
## Proportion of Variance 0.2241  0.1017  0.08511  0.07383  0.06957  0.05427  0.05079
## Cumulative Proportion 0.2241  0.3258  0.41089  0.48472  0.55429  0.60856  0.65935
##              PC8      PC9      PC10     PC11     PC12     PC13     PC14
## Standard deviation  0.93867  0.92718  0.8807  0.85563  0.81030  0.73429  0.68600
## Proportion of Variance 0.04895  0.04776  0.0431  0.04067  0.03648  0.02995  0.02614
## Cumulative Proportion 0.70830  0.75606  0.7992  0.83983  0.87631  0.90626  0.93240
##              PC15     PC16     PC17     PC18
## Standard deviation  0.65207  0.58828  0.50527  0.43607
## Proportion of Variance 0.02362  0.01923  0.01418  0.01056
## Cumulative Proportion 0.95603  0.97525  0.98944  1.00000
```

2. You may also notice that this is always ordered from most proportion of variance on the left (thus is PC1 defined) to our final principal component on the right. What percent of variance is explained by just PC1? What percent by the final PC? Is the attributable portion of the variance in PC1 high, medium, or low based on some quick research or prior knowledge?

Your answer here. ##PC1 explains about 22.4% of the variance, while the final principal component explains only about 1.1% of the variance. This amount would make PC1 to be moderately high, since nearly a quarter of the variation in the dataset is captured by a single component.

3. We use scree plots to graphically determine how many principal components to retain and how many principal components best explain the variance of the data. The *x*-axis displays the principal components (PC1, PC2, etc.), while the *y*-axis shows the eigenvalues (or proportion of variance explained) for each component. The **elbow method** we discussed in class is the point where the slope starts to flatten or the point at which the percentage of variance explained seems to level off. If we were doing this for feature reduction, we might think of this as approximately how many principal components would be needed to explain the total variation in the training data. However, it is a **heuristic** method, so it can sometimes be a little uncertain from the plot alone what the optimal number of PCs is. Let's take a look at the top-15 PCs on a scree plot now:

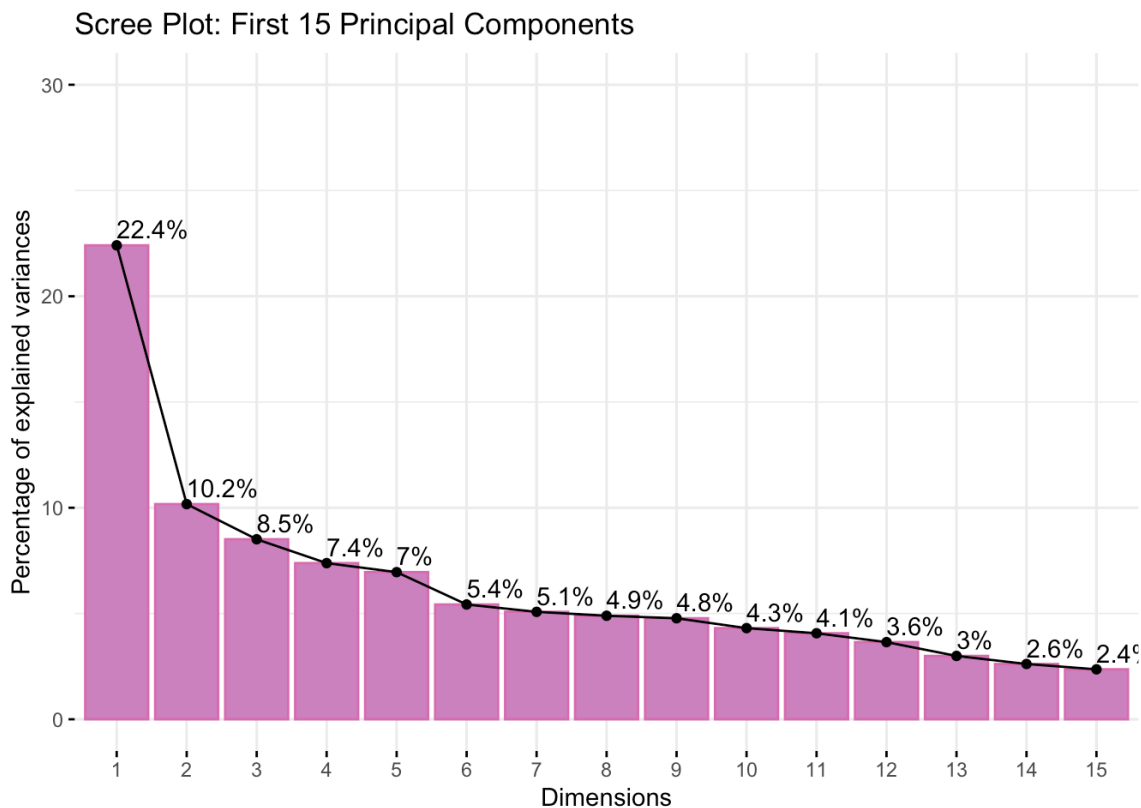


Figure 10. Scree Plot of First 15 Principal Components

How many PCs appear optimal to you based on the scree plot?

Your answer here. ##Based on the scree plot around 5 principal components appear to be the most optimal cutoff. The largest drop in explained variance happens in the first few components, and after about PC5 the curve starts to flatten. While additional PCs still explain some variance, the rate of improvement becomes much smaller after this point.

4. People often mis-write scree (I've even accidentally done it) as SCREE, as if it's an acronym like AUROC or ROC or even PCA. But it's not. Why is it called a scree plot? (Hint: It may not be what you expect!)

Your answer here. ##A scree plot is called a "scree" plot because it resembles the shape of a mountain slope with loose rocks, or scree, at the bottom. In PCA, the first few principal components usually explain much more variance, creating a steep drop, while later components flatten out and contribute smaller amounts of variance. That flatter section resembles the pile of loose rock at the base of a mountain, ### 5.1.2 Variable loadings and contributions

Next, let's explore how much the variables are contributing (this is based on the weight of their rotations). In PCA, we are interested in the **loadings**, i.e., the linear combination weights (coefficients) whereby unit-scaled components define or "load" a variable. Loadings help us interpret principal components.

You may have noticed we used the `prcomp()` function above; `prcomp()` will return the loadings in the variable `$rotation`, which contains a matrix of variable loadings. These loadings have been determined from the eigenvectors, which without getting into what that is (because we'd have to take a departure into linear algebra), suffice it to say that they are how we are measuring the direction and magnitude during each subsequent rotation of the principal components as we measure the variance explained.

Additionally, we are going to color this by our grouping variable and key predictor in the dataset, `Hospital.return.days.for.pneumonia.patients`. Recall that when we explored this earlier this variable is the performance of each hospital in terms of its pneumonia-related return-to-hospital days relative to the national average. In other words, hospitals that were “Worse than average” had readmitted pneumonia patients with longer-than-average stays.

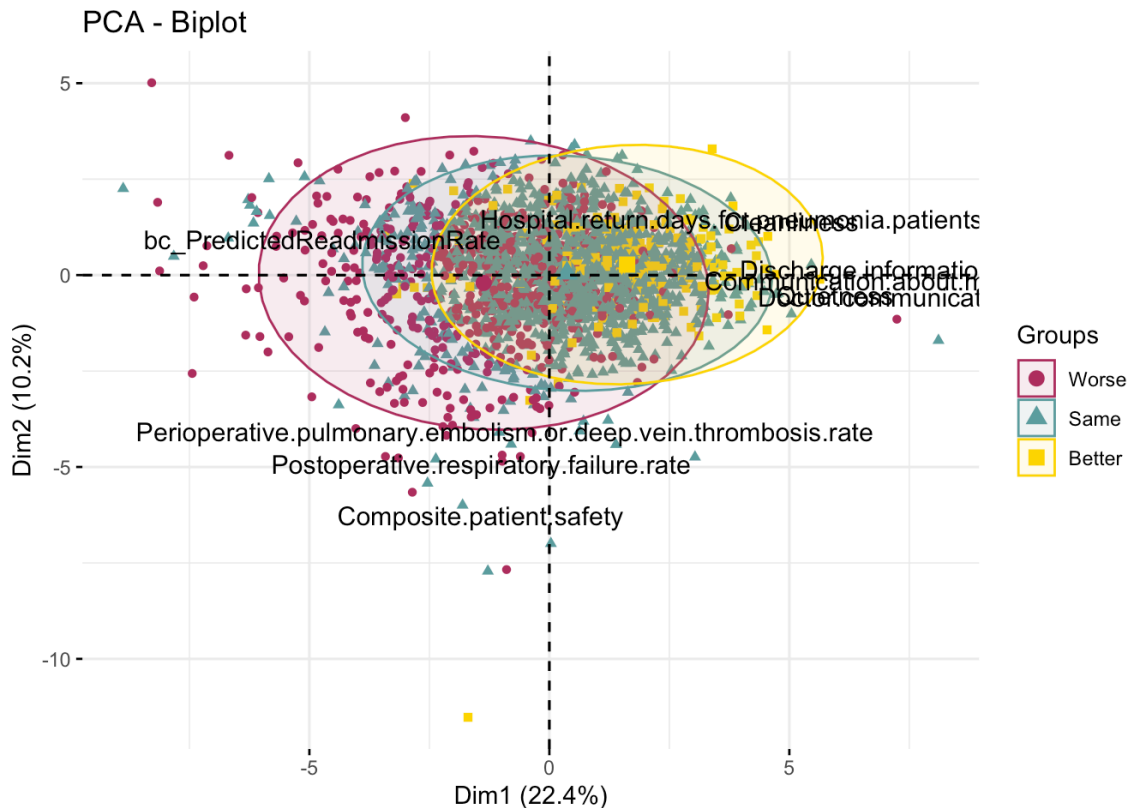


Figure 11. PCA Biplot Showing the Top Feature that Contributes to Explained Variance

This biplot (Figure above) shows that, on principal components 1 and 2 (the PCs that account for the **most** variance in the training data) there is subtle but noticeable grouping of hospitals in terms of how long their readmitted pneumonia patients stayed in the hospital. I have chosen to display the **#1** feature that contributes to that variance, `Communication.about.medicines`.

Each quadrant on a PCA biplot corresponds to a different directional influence of the original variables. E.g., because it is in the top-right quadrant I where both PC1 and PC2 are positive, this suggests that communication about medicines are positively associated with variables that point into the same quadrant. Since it seems to correspond with where the “better” hospitals are, this suggests that these hospitals are made somewhat distinctly “better” at readmitted hospital stay lengths because of correlations with communication about medicines. (**NOTE:** If you look at the 2025 data, the biplot is effectively inverted but the conclusions seem to be the same in 2025 as well).

A general rule of thumb is:

1. Top-right Quadrant I

- PC1 and PC2 are both positive
- Thus points in this quadrant are positively associated with variables pointing into this quadrant
- Indicates high values for variables with arrows in this quadrant

2. Top-left Quadrant II

- PC1 is negative, PC2 is positive
- Thus points here are associated with high values of variables pointing to the left and top
- These observations contrast with those in the right-side quadrants on PC1

3. Bottom-left Quadrant III

- PC1 and PC2 are both negative
- Thus points here are negatively associated with variables pointing into the other quadrants
- These points could represent a distinct subgroup or pattern with behavior opposite to other points.

4. Bottom-right Quadrant IV

- PC1 is positive, PC2 is negative
- Thus points here are associated with variables pointing *into* this quadrant
- May indicate a contrast with Quadrant II on PC2

Question 22: [1 point]

Play around with the graphical parameters in the biplot, especially those I've marked with comments. Try adding, for example, more variables to display rather than just the top contributor. **NOTE** that if you find variables in the same quadrant together, this means that they are correlated. What variable(s) can you find that **contrast** with Communication.about.medicines ?

Your answer here. ##In the biplot, Communication.about.medicines is more toward the left/center of the plot, while variables like Cleanliness, Discharge.information, Quietness, and Doctor.communication appear more toward the right side. Because these variables are pointing in a different direction from Communication.about.medicines, they appear to contrast with it in the PCA, suggesting hospitals that align more strongly with communication about medicines may not be the same hospitals that align most strongly with cleanliness, discharge information, quietness, or doctor communication. Since the variables are still somewhat close to the center and there is overlap between groups

Now, look at the contributions to PCs 1 and 2 (Figure Below) from the variables in the training dataset. Anything above the dotted red-line contributes **significantly** to the overall explanation of variance, and anything below the dotted line does not. Does anything about this plot stand out to you?

HINT 1: What do the top contributing variables all have in common? In other words, did they all come from the same data source?

HINT 2: Where does the target, PredictedReasmissionRate , fall among the significant variables?

Your answer here.

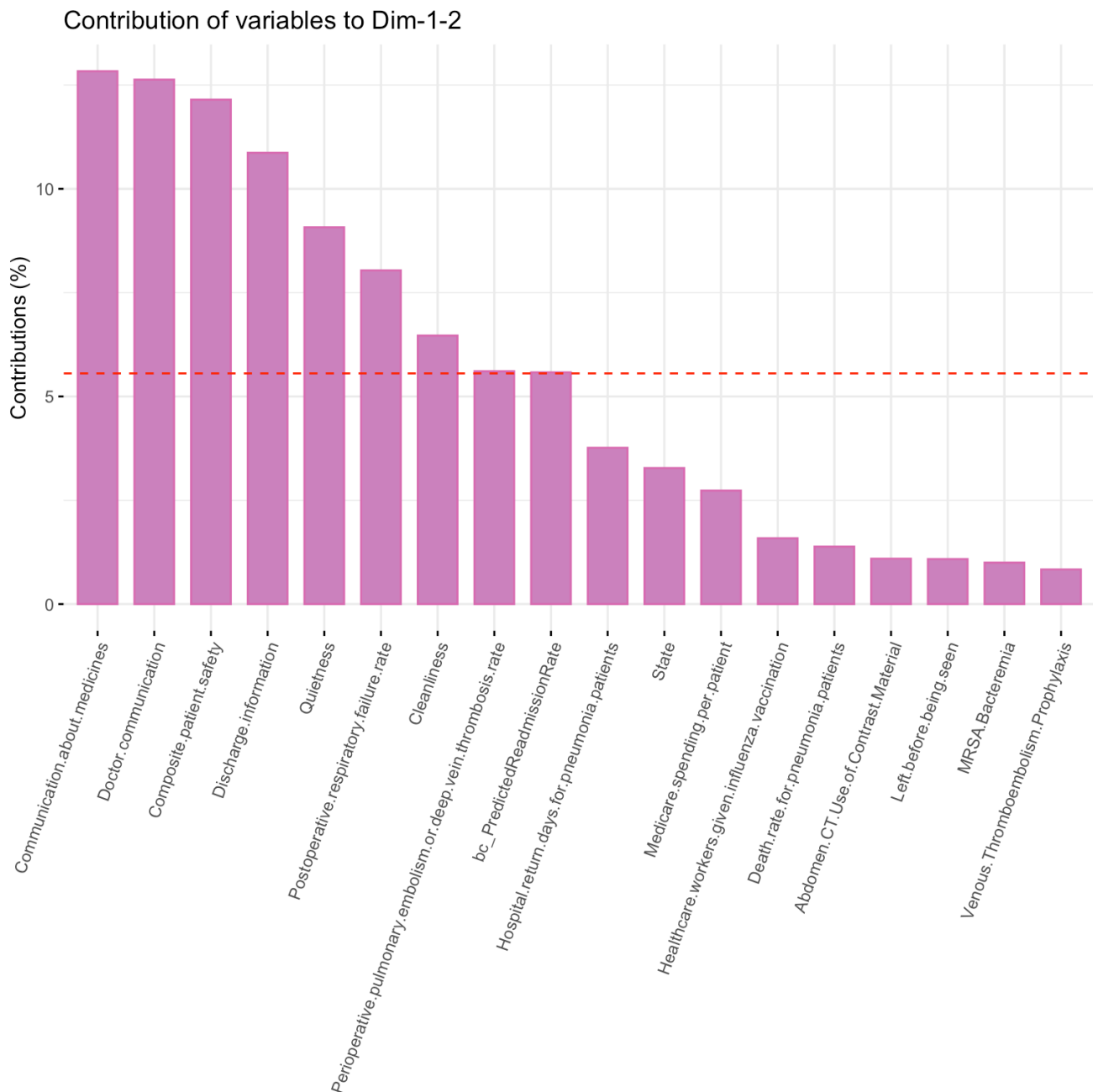


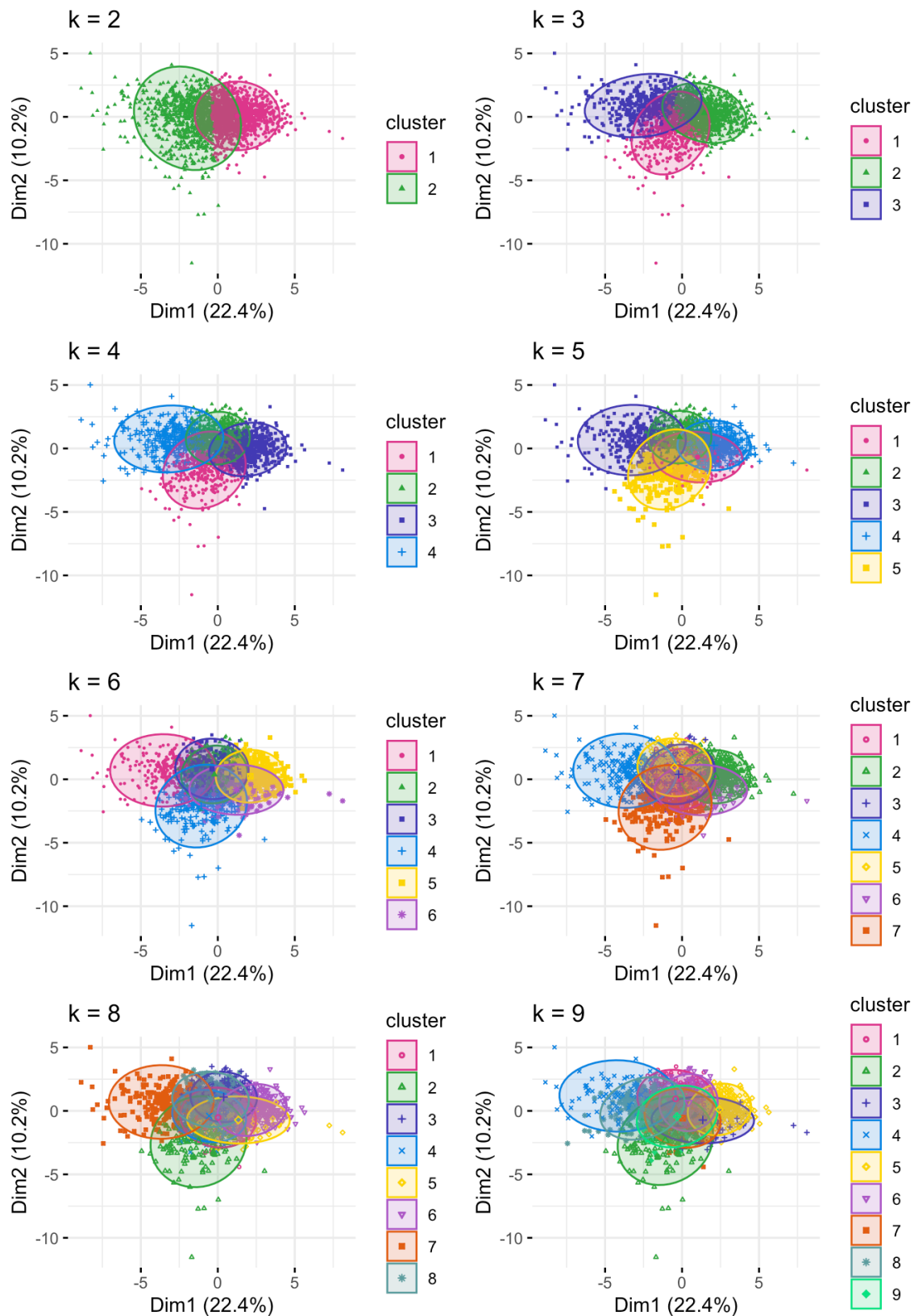
Figure 12. Variables that Significantly Explain Variance.

5.2 k -means Clustering

k -means is a centroid-based clustering algorithm, where a “centroid” is the geometric center of an object often measured by Euclidean distance. In the algorithm, the distance between each data point and a centroid is calculated by randomly grabbing data; these distances are then used to assign the data point to a cluster. The goal is to identify the optimal k number of groups in a dataset by minimizing the distance of each datapoint to a respective centroid.

5.2.1 Visualizing Clusters

Although multiple methods exist to determine the optimal number of clusters, one commonly employed is **heuristic**, i.e., it isn't really computational. You will either choose the optimal clusters based on a set of options of k graphically or you can once again employ the “elbow” method again, as you did with PCA. Let's start by visually inspecting the effect of different k clusters on PC1 and PC2 of the dataset. I have chosen to test k groups of 2 through 9.

Figure 13. Biplots showing possible k clusters**Question 23A:** [0.5 points]

Which number of clusters, k , do you think has the best explanatory power? Why? Is it hard to tell?

Your answer here ##Based on the cluster plots, $k = 4$ has the best explanatory power. With only 2 or 3 clusters, the data still appears overly broad and several observations overlap heavily, which may oversimplify patterns in the dataset. By $k = 4$, the clusters begin to show more distinct structure while still remaining interpretable. Once k increases to 6–9, the clusters start to feel more fragmented and difficult to distinguish

5.2.3 Within-sum-of-squares (WSS) elbow method

Next, let's employ the 'elbow' method again, this time to look at when the **within sum of squares** drops off rather than the percent variance, as was done in PCA.

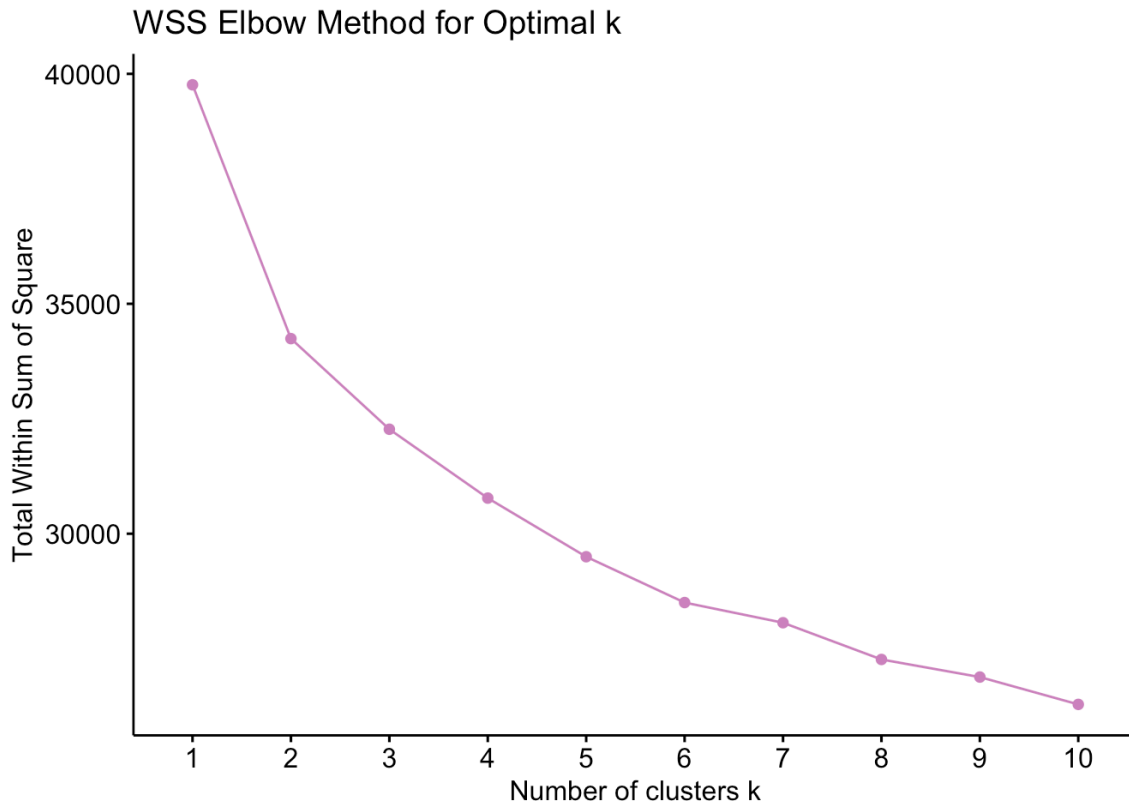


Figure 14. WSS Scree plot for k-means

Question 23B: [1 point]

What is the **within sum of squares** and what exactly is it measuring here? By the elbow method, which k is the optimal number of clusters? Did it agree with your choice from the other heuristic method?

Your answer here ##The within sum of squares (WSS) measures how closely observations within a cluster are grouped around the center of that cluster (centroid). In this case, it is helping measure how similar hospitals are to others within the same cluster based on the PCA-transformed features. Lower WSS values generally suggest tighter, more internally consistent clusters. However, one thing to keep in mind is that WSS naturally decreases as more clusters are added, since creating smaller groups will almost always reduce variation within a cluster.

Looking at the elbow plot, $k = 4$ appears to be the most reasonable choice because this is where the curve starts to level off and the reduction in WSS becomes less substantial

5.2.3 Silhouette method

Lastly, we will apply the **silhouette method**, which measures the average silhouette width, a combination of (1) how similar any given point is to its own cluster (also called **cohesion**) and (2) how different that same point is from other clusters (also called **separation**). Silhouette scores can range from $-1 \rightarrow +1$, with larger values indicating better clustering (i.e., more cohesive clusters that are separated from each other). Thus, a silhouette score of 1 indicates **perfect separation and cohesion** whereas a silhouette of 0 indicates overlapping clusters.

What we can do is iteratively test our possible k values of 2 to 9 and a silhouette score will be calculated for each k tested. The highest silhouette score indicates the optimal k .

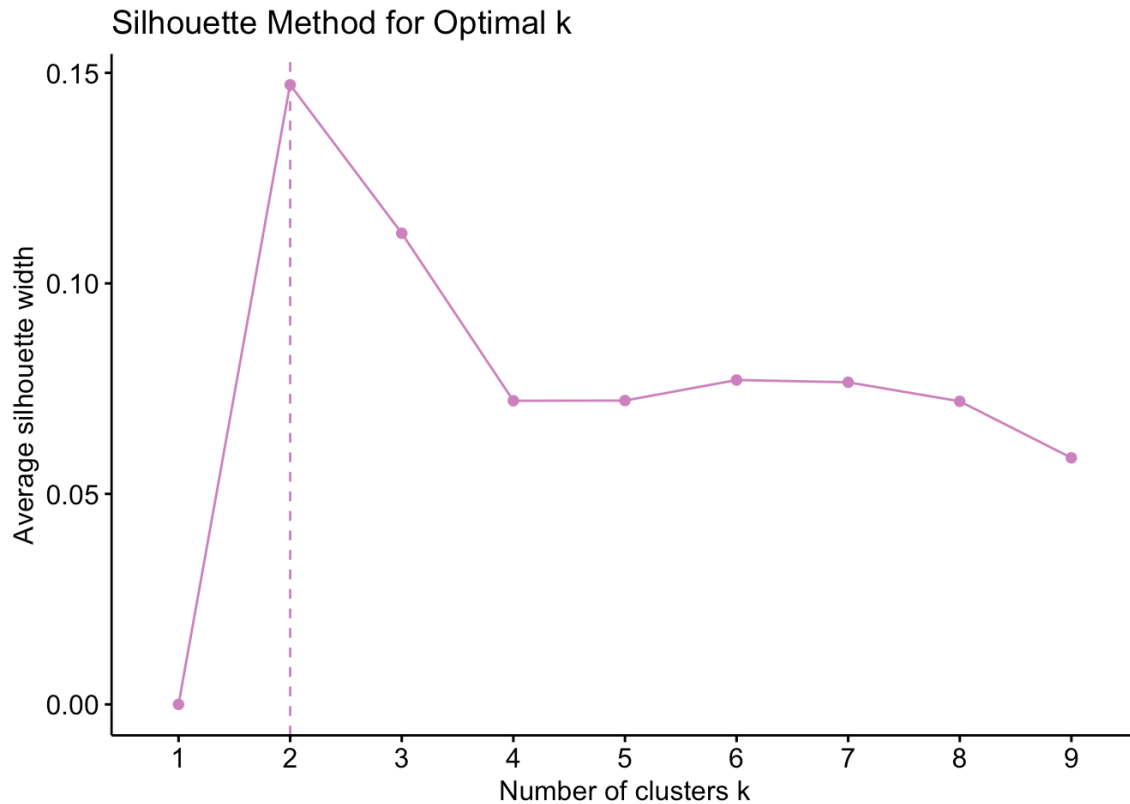


Figure 14. Silhouette Plot for k-means

Question 23C: [0.5 points]

Are you surprised at the optimal k from the silhouette method? Does it seem to agree with what you've seen with the WSS elbow method? (**Hint:** agreement generally suggests evidence that you have found the optimal k).

Your answer here ##Yes, we're somewhat surprised because the silhouette method suggests that $k = 2$ is the best number of clusters since it has the highest average silhouette width, meaning the observations are more clearly separated when divided into two groups. This differs from the elbow method and visual cluster interpretation, where $k = 4$ appeared more useful and meaningful for interpretation. So, while $k = 2$ may be statistically cleaner in terms of separation, $k = 4$ may still be more practical if the goal is to better understand more detailed hospital group patterns.

Lastly, let's extract the optimal k from the silhouette method.

```
## Extract optimal k from silhouette method; first get the cluster data
clusterData <- fviz_nbclust(readyTrain, kmeans, method = "silhouette")$data
## Then extract the maximum y
k <- as.numeric(clusterData$clusters[which.max(clusterData$y)])
```

5.3 Segmentation Analysis

As we said before, the goal of segmentation analysis is to broadly cluster our hospitals by their features to help us characterize them.

Question 24: [1 point]

Let's explore the clusters - segments - of the hospitals based on their average values from the **original** dataset (we start from `imputedTrain`, which hadn't yet been transformed, centered, or scaled, BUT the multicollinear variables also haven't been dropped either). Your task is to add comments to this code chunk.

```

# Run k-means clustering using the selected number of clusters (k).
## This groups hospitals into segments based on similar characteristics.
kmeansResult <- kmeans(readyTrain,
                        centers = k,      ## Use the selected number of clusters (k).
                        nstart = 50,     ## Test multiple starting points to improve cluster
ring stability.
                        iter.max = 1000, ## Allow enough iterations for the algorithm to co
nverge.
                        algorithm = "MacQueen")

## Return to the original imputed training dataset so cluster labels
## can be interpreted using more meaningful hospital variables.
kmeansTrain <- imputedTrain %>%

  ## Rename the pneumonia return-days variable for readability.
  rename(Pneumonia.Hospital.Return.Days =
    `ComparedToNational_Hospital return days for pneumonia patients`) %>%

  ## Keep important PCA/contribution variables and remove
  ## redundant or unnecessary variables.
  select(-any_of(vifOver5),
    -contains("Compared"),
    -contains("PaymentCategory"),
    -SurveyResponseRate) %>%

  ## Add cluster labels to hospitals and recode pneumonia return days
  ## into numeric values for easier analysis.
  mutate(
    Cluster = kmeansResult$cluster,
    Pneumonia.Hospital.Return.Days =
      as.numeric(if_else(Pneumonia.Hospital.Return.Days == "1", 1,
        if_else(Pneumonia.Hospital.Return.Days == "0", 0, -1)))
  ) %>%

  ## Standardize variable names for cleaner analysis.
  clean_names()

## List the important PCA variables that we want to compare across clusters.
pcaImportantVars <- factor(c(
  "Communication.about.medicines",
  "Doctor.communication",
  "Composite.patient.safety",
  "Discharge.information",
  "Quietness",
  "Postoperative.respiratory.failure.rate",
  "Cleanliness",
  "Periop.Rate.Of.Embolism.Or.Thrombosis",
  "Predicted.Readmission.Rate",
  "Pneumonia.Hospital.Return.Days"
))

## Convert variable names to lowercase.
pcaImportantVars <- tolower(pcaImportantVars)

## Replace periods with underscores and format names for presentation.
pcaImportantVars <- gsub("\\.", "_", pcaImportantVars) %>%

```

```

to_any_case("title")

## Reshape the data into long format so important variables
## can be plotted and compared across clusters.
kmeansTrain %>%

## Rename variables for readability.
rename(
  Predicted.Readmission.Rate = predicted_readmission_rate,
  Periop.Rate.Of.Embolism.Or.Thrombosis =
    perioperative_pulmonary_embolism_or_deep_vein_thrombosis_rate
) %>%

## Format variable names for presentation.
clean_names(case = "title") %>%

## Keep only important PCA-related hospital measures.
select(Cluster, any_of(pcaImportantVars)) %>%

## Convert the data into long format for plotting.
pivot_longer(-1,
  names_to = "Measure",
  values_to = "Value") %>%

## Preserve the ordering of variables in the final visualization.
mutate(
  Measure = factor(
    Measure,
    levels = to_any_case(c(
      "Communication.about.medicines",
      "Doctor.communication",
      "Composite.patient.safety",
      "Discharge.information",
      "Quietness",
      "Postoperative.respiratory.failure.rate",
      "Cleanliness",
      "Periop.Rate.Of.Embolism.Or.Thrombosis",
      "Predicted.Readmission.Rate",
      "Pneumonia.Hospital.Return.Days"
    )),
    case = "title")
  )
) %>%

## Create boxplots comparing hospital measures across clusters.
ggplot(aes(x = Cluster,
  y = Value,
  fill = as.factor(Cluster))) +

geom_boxplot(alpha = 0.75) +

## Apply custom colors to distinguish clusters.
scale_fill_manual(values = c(skittles[1], skittles[6])) +

## Add labels and title to improve readability.
labs(title = "Cluster Means",
  x = "",

```

```

fill = "Cluster") +

## Format the appearance of the plot.
theme_classic() +
theme(
  legend.position = "bottom",
  strip.text = element_text(size = 8),
  legend.text = element_text(size = 8),
  legend.title = element_text(size = 10),
  title = element_text(size = 12, color = "maroon")
) +

## Show each hospital measure in a separate panel.
facet_wrap(~ Measure,
  scales = "free_y",
  ncol = 2)

```

Cluster Means

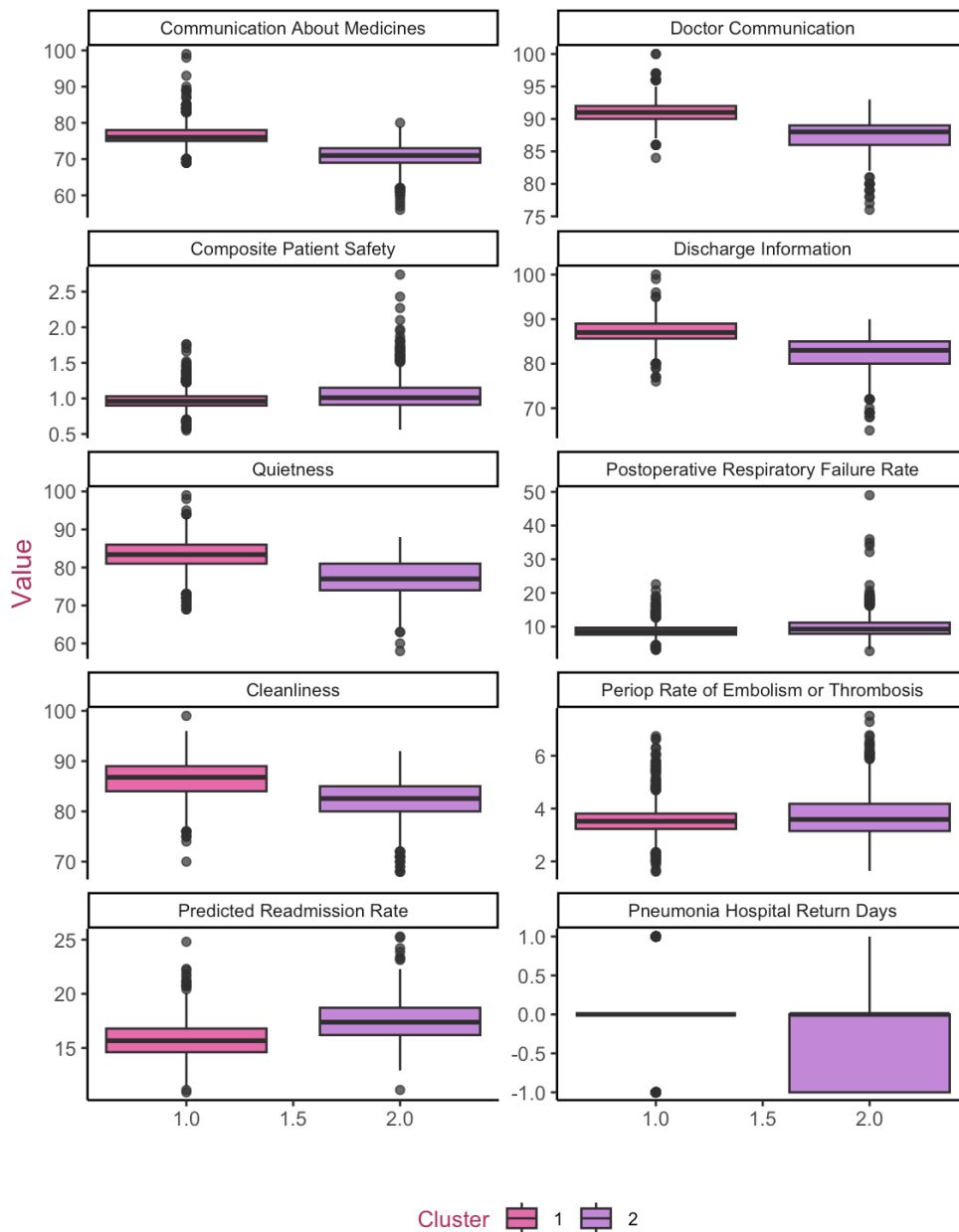


Figure 16. Boxplot Comparisons of Important Variables by Cluster

```
#### Create a summary table showing the mean and median values
#### of important hospital variables by cluster.
kmeansTrain %>%

## Rename variables for readability.
rename(
  Predicted.Readmission.Rate = predicted_readmission_rate,
  Periop.Rate.Of.Embolism.Or.Thrombosis =
    perioperative_pulmonary_embolism_or_deep_vein_thrombosis_rate
) %>%

## Format names for presentation.
clean_names(case = "title") %>%

## Keep important cluster variables.
select(Cluster, any_of(pcaImportantVars)) %>%

## Summarize cluster means and medians.
tbl_summary(
  by = "Cluster",
  statistic = list(all_continuous() ~ c("{mean} ({median})"))
) %>%

## Rename table headers for readability.
modify_header(
  label ~ "Variable",
  all_stat_cols() ~ "**Cluster {level}**"
) %>%

## Add a descriptive caption to the summary table.
modify_caption(
  "Table 8. Summary Statistics of Important Cluster Variables"
)
```

Table 8. Summary Statistics of Important Cluster Variables

Variable	Cluster 1 [†]	Cluster 2 [†]
Communication About Medicines	76.5 (76.0)	70.9 (71.0)
Doctor Communication	91.03 (91.00)	87.68 (88.00)
Composite Patient Safety	0.98 (0.96)	1.06 (1.01)
Discharge Information	87.0 (87.0)	82.3 (83.0)
Quietness	83.2 (83.4)	77.1 (77.0)
Postoperative Respiratory Failure Rate	8.69 (8.53)	9.93 (9.25)
Cleanliness	86.4 (86.8)	82.1 (82.6)
Periop Rate of Embolism or Thrombosis	3.55 (3.52)	3.73 (3.59)
Predicted Readmission Rate	15.77 (15.67)	17.57 (17.38)
Pneumonia Hospital Return Days		
[†] Mean (Median); n (%)		

Variable	Cluster 1 ¹	Cluster 2 ¹
-1	188 (14%)	430 (50%)
0	976 (72%)	408 (47%)
1	185 (14%)	23 (2.7%)
¹ Mean (Median); n (%)		

HINT: If these are back to the *original* values, this is more interpretable for our stakeholder. We will now read this just like a typical boxplot; for example, median (the middle line) Doctor Communication seems higher in **Cluster 1** than it does in **Cluster 2** with only some overlap in the distributions (the boxes and whiskers don't really overlap each other). The table confirms this; the median Doctor Communication score is 91% in Cluster 1 but only 88% in Cluster 2.

Question 25: [2 points]

Notice, for example, that Cluster 1 hospitals seem to include the hospitals that had the **lowest** pneumonia-related readmission rates, with the average rate ~2% higher in Cluster 2 hospitals. Thus, for our stakeholder, we might decide to rename Cluster 1 as “Highest Performing Hospitals” or something along those lines. Notice also that this cluster had more **below** national average return hospital days (a “1”). This means that they had more readmitted pneumonia patients who spent less time in the hospital upon readmission than the national average.

Look at how else you might broadly characterize - or segment - these hospitals. Your job is to make a table for your client, summarizing each of the two clusters with FOUR primary criteria. Make sure to give each cluster an informative name as I did; you’re welcome to rename Cluster 1 if you think of a catchier name than I chose!

After you come up with names of the two clusters, you will then come up with at least FOUR characteristics per cluster summarizing what you see in either the boxplots or table. You may choose to focus on different attributes for different clusters. Fill in the code for the table below. I start you out with two examples for Cluster 1, but feel free to add more!

Table 9. Hospital segmentation analysis: two broad hospital groupings identified.

Top Defining Attributes	Description
1: Highest Performing Hospitals	
Predicted pneumonia-related hospital readmissions	These hospitals have lower readmission rates, suggesting stronger pneumonia-related care outcomes.
Return hospital days due to pneumonia	Readmitted patients spend fewer days in hospital relative to national benchmarks, suggesting more efficient recovery or care coordination.
Doctor communication	Patients report stronger physician communication, suggesting clearer provider-patient interactions and care coordination.
Discharge information	Patients receive stronger discharge guidance, suggesting hospitals better prepare patients for post-hospital recovery.
2: Lower Performing Hospitals	
Higher predicted pneumonia-related hospital readmissions	These hospitals experience higher readmission rates, suggesting weaker pneumonia-related outcomes or greater risk of return visits.
Return hospital days due to pneumonia	Patients are more likely to experience above-national-average hospital return patterns following pneumonia care.
Doctor communication	Patients report weaker physician communication scores, suggesting less effective provider-patient communication.

Top Defining Attributes**Description**

Discharge information

Patients report weaker discharge preparation, suggesting opportunities for improved post-hospital recovery guidance.

5.4 Final Remarks on Segmentation

We don't technically have to stop our segmentation analysis here, but we will so that we can focus on other analyses. One thing we might choose to do in the future, for example, is to choose a classification machine learning algorithm to test the predictions based on these clusters.

Question 26: [1 point]

Make a recommendation for our client for 1-2 machine learning analyses **your team** would choose to use to test whether these clusters can be used for robust predictions of hospital performance for pneumonia patients. How would you know whether or not your predictions were robust? What would you look for or compare?

Your answer here. ##Team Alpha would recommend starting with an Ordinary Least Squares (OLS) regression followed by an Elastic Net model to test whether these hospital clusters can be used to make reliable predictions about pneumonia-related hospital performance. We would start with OLS because it gives us a simple and interpretable baseline for understanding whether hospital cluster membership is associated with outcomes such as predicted readmission rates or pneumonia-related return patterns. Since many hospital quality measures overlap with one another (for example, doctor communication, discharge information, and patient safety), we would also use Elastic Net because it can better handle correlated predictors and help identify which hospital characteristics are most important for prediction. To determine whether our predictions are robust, Team Alpha would compare model performance on both training and testing data to make sure the model performs consistently and is not just memorizing patterns in one dataset. We would compare measures such as prediction error and overall model fit, while also evaluating whether hospital cluster membership continues to meaningfully predict pneumonia-related outcomes across models. If the models perform similarly on new data and cluster membership remains informative, this would suggest the hospital groupings are useful for predicting performance and may provide meaningful insights for healthcare stakeholders.

6 Supervised Learning Methods

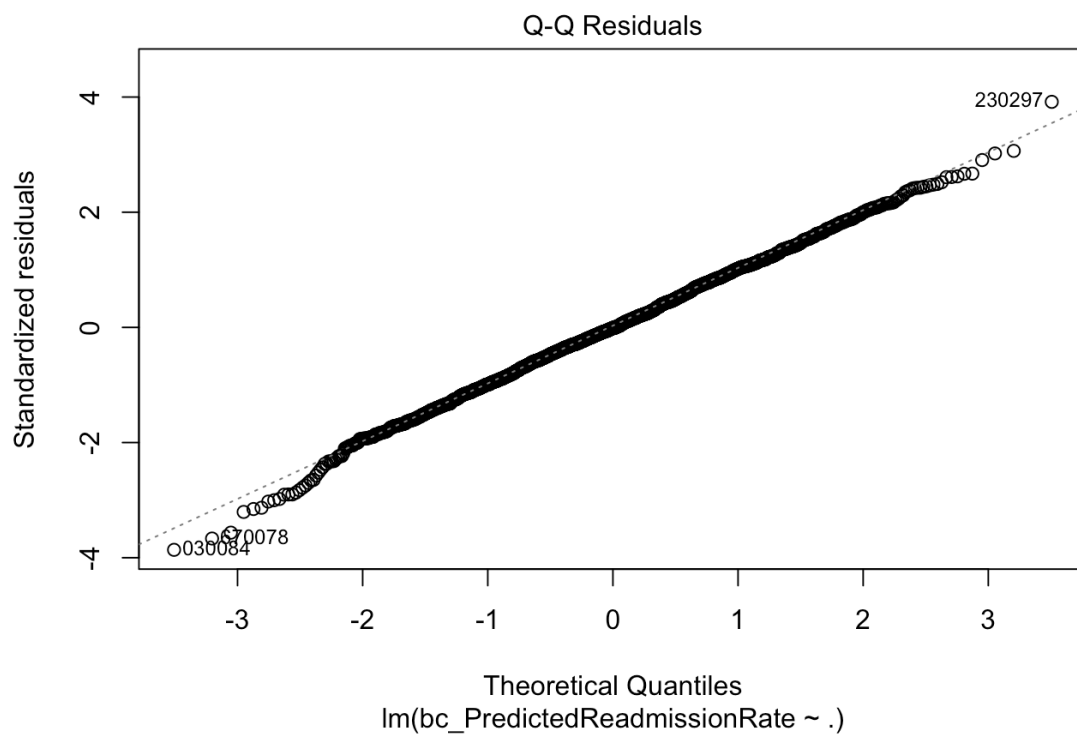
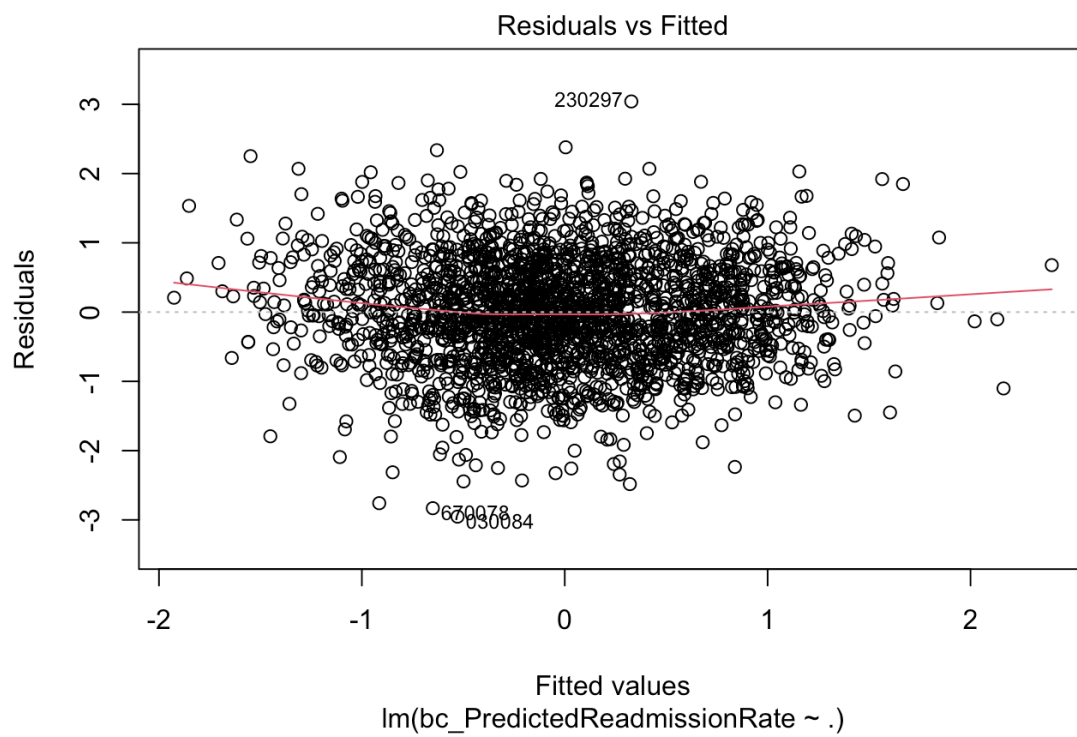
We are going to perform two supervised learning methods: an Ordinary Least Squares (OLS) regression followed by an Elastic Net.

6.1 Ordinary Least Squares (OLS) regression

Earlier in Section 4 when we assessed multicollinearity, we fit an OLS regression model so that we could calculate variance inflation factors (VIFs). However, because OLS regression is so incredibly sensitive to multicollinearity, we should re-run it now that we've removed some of the most redundant variables.

Let's model that now, using the `readyTrain` dataset and plot some diagnostic plots to assess the key assumptions of an OLS linear regression. The four plots are diagnostic plots for the primary assumptions of a linear regression:

1. **Linearity** between the outcome and the predictors
2. **Normally distributed** residuals
3. **Homoscedasticity** (equality of variances) in the residuals
4. **No high leverage/influence points** (no significant outliers)



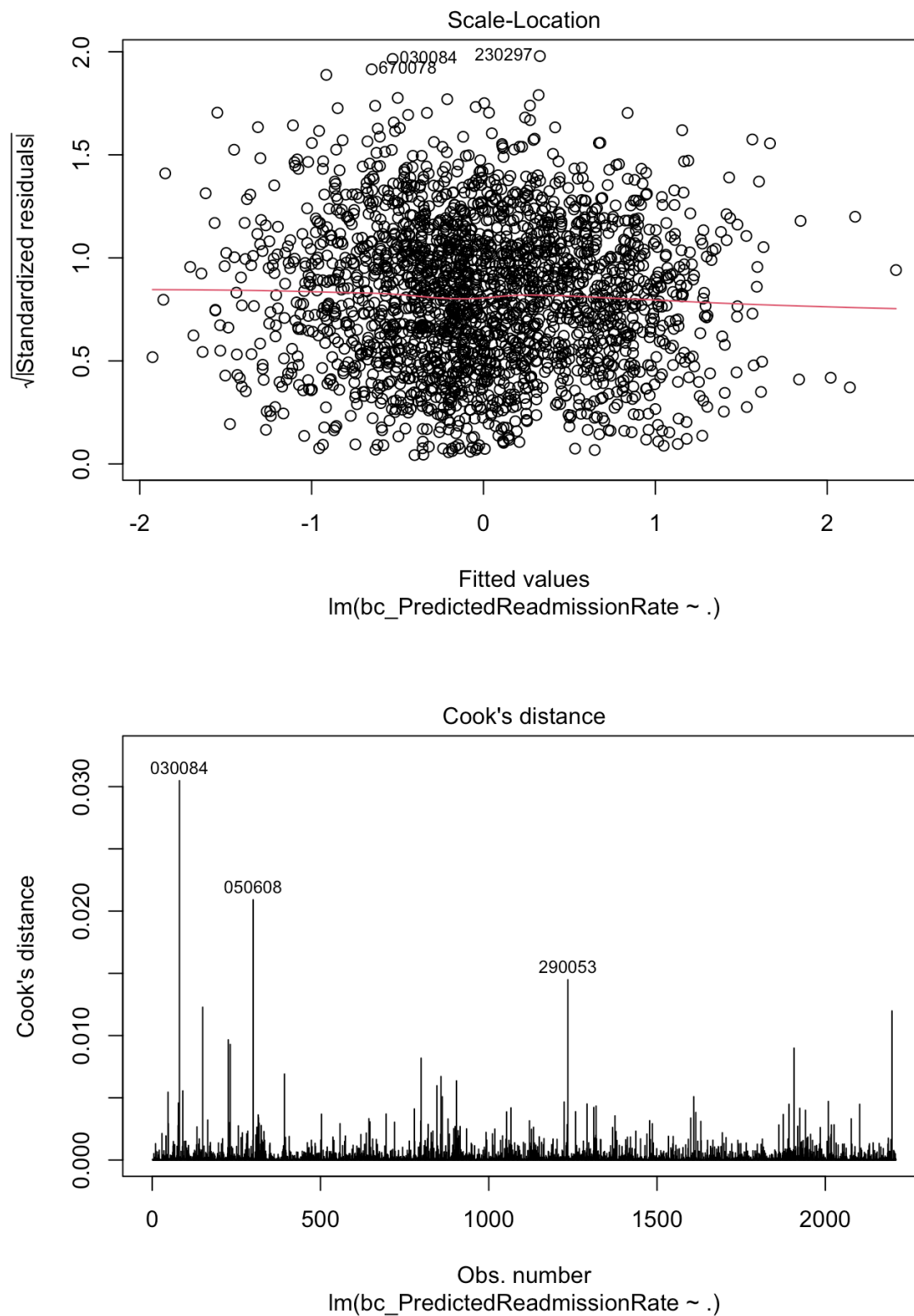


Figure 17. OLS Regression Assumption Diagnostic Plots

Question 27A: [1 point]

What is your general assessment of our OLS assumptions? Met or unmet? Why?

Your answer here. ##Overall, the OLS assumptions were reasonably met. The residuals vs fitted plot suggests the linearity assumption is mostly supported because the residuals appear randomly scattered around zero, even though there is slight curvature at the extremes. The Q-Q plot and Shapiro-Wilk test ($p = 0.067$) suggest the residuals are reasonably close to normal since the points mostly follow the reference line and the p-value is above 0.05. The scale-location plot also suggests relatively constant variance because the spread of residuals appears fairly stable across fitted values, although there is some minor variation. Finally, Cook's distance identified a few observations, but none appear severe enough to suggest major concerns. Overall, the assumptions appear sufficiently met for modeling purposes.

Note: If you are not entirely sure about the Q-Q plot, you can run a Shapiro-Wilk test on the residuals using the following code:

```
shapiro.test(residuals(mod))
```

```
##
## Shapiro-Wilk normality test
##
## data: residuals(mod)
## W = 0.99862, p-value = 0.06735
```

The **most important** OLS assumption for Elastic Net is **linearity of the relationship between predictors and the response**. The other, stricter OLS assumptions don't matter as much because of regularization.

Do you find support for linearity here? Why or why not?

Your answer here. ##Yes we find reasonable support for linearity in this model. Based on the residuals vs fitted plot, the residuals appear to be fairly randomly distributed around zero without a strong curved pattern, which suggests a generally linear relationship between the predictors and the outcome. While the smoothing line shows some slight deviation at the extremes, the pattern is still relatively stable overall. Because there is no strong systematic trend in the residuals, we believe the linearity assumption is reasonably supported.

2. Let's now assess how well our model fits with the adjusted R^2 .

```
## [1] 0.39427
```

Question 27B: [1 point]

It is a surprisingly high (although it isn't stellar). You may need to do some outside research, but explain in a sentence or two why, generally speaking, we use **adjusted R^2** and not the unadjusted one (confusingly labeled as multiple r -squared in the `summary()` output below).

Your answer here. We use adjusted R^2 instead of multiple R-squared because adjusted R^2 accounts for the number of predictors in the model, making it a more realistic measure of model fit. Multiple R-squared tends to increase as more variables are added, even if those variables do not meaningfully improve the model.

3. Find the best model.

Use the `step()` function to do a backward regression to find our **most parsimonious** model. Stepwise regression drops terms that are not significant or important, thereby only keeping the features that contribute to the overall explanation of variance in predicted readmission rate.

```
## Perform a backward, stepwise regression to find the most parsimonious model
bestMod <- step(mod,
  direction = "backward",
  trace = FALSE)
```

Lastly, let's take a peek at the results. We will use a package called `stargazer` to help us visualize the results in a nicely organized way.

```
## Print a table using stargazer
stargazer(bestMod,
  type = "text",
  title = "Table 10. Parsimonious OLS Regression Results.")
```

Table 10. Parsimonious OLS Regression Results.

Dependent variable:

bc_PredictedReadmissionRate

```
MRSA.Bacteremia 0.039** (0.017)
Abdomen.CT.Use.of.Contrast.Material 0.045*** (0.017)
Death.rate.for.pneumonia.patients -0.154*** (0.017)
Medicare.spending.per.patient 0.167*** (0.018)
Healthcare.workers.given.influenza.vaccination 0.044** (0.018)
Left.before.being.seen 0.062*** (0.017)
Venous.Thromboembolism.Prophylaxis 0.101*** (0.018)
Doctor.communication -0.074*** (0.027)
Discharge.information -0.053** (0.024)
Cleanliness -0.053*** (0.020)
Quietness -0.077*** (0.022)
Hospital.return.days.for.pneumonia.patients -0.396*** (0.018)
State 0.110*** (0.019)
Constant 0.000 (0.017)
```

Observations 2,210

R² 0.398

Adjusted R² 0.395

Residual Std. Error 0.778 (df = 2196)

F Statistic 111.824*** (df = 13; 2196)

===== Note: $p < 0.1$; $p < 0.05$; $p < 0.01$

Lastly, let's assess how good a predictive model our OLS regression model is. We will use the `predict()` function to first fit our most parsimonious model `bestMod` to the test data:

```
## Make the predictions
predictions <- predict(bestMod, newdata = readyTest)
```

The Root Square Mean Error (*RMSE*) is the square root of the average of the squared differences between predicted and actual value, such that $RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$ where y_i is each observed value, $\hat{y}_i$ is each corresponding predicted value, and n is the number of observations. **NOTE:** Recall that a residual is $y_i - \hat{y}_i$ - the distance between every point and its predicted value! It's also important to realize that the *RMSE* gives more weight to large errors because of squaring. Thus, it's sensitive to outliers.

```
## Calculate the RMSE as the square root of the mean of the residuals, squared
RMSE <- sqrt(mean((readyTest$bc_PredictedReadmissionRate - predictions)^2))
RMSE
```

```
## [1] 0.8102472
```

Question 27C: [1 point]

Can you interpret what the RMSE tells your client here? Why or why not? What about the R^2 ?

Hint 1: Remember that the outcome here is Box-Cox transformed. Would you need to undo the transformation to be able to assess what the RMSE (or R^2) is actually telling us? Recall that the Box-Cox $y^{\hat{\lambda}} = \frac{1}{y^{\lambda}}$ where our $\lambda = -0.3$ AND we also centered and scaled these data.

Hint 2: Is it practical to undo both transformations? How does that hinder our ability to make an interpretation for our client?

Your answer here. ##While the RMSE is about 0.81, it is difficult to explain exactly what this means to a client because the outcome variable was Box-Cox transformed and then centered and scaled before running the model. Therefore, the prediction error is not in the original readmission rate units anymore, so we would need to undo those transformations to make the number more meaningful in a real-world sense. The R^2 of about 0.398 tells us the model explains around 40% of the variation in the outcome, but this should also be interpreted carefully since it is based on the transformed version of readmission rate and not the original scale.

Lastly, let's graph the effect predictive ability of our OLS model by comparing the relationship between the **true** values of `bc_PredictedReadmissionRate` from the `readyTest` dataset to the predictions we made using the model on the testing data.

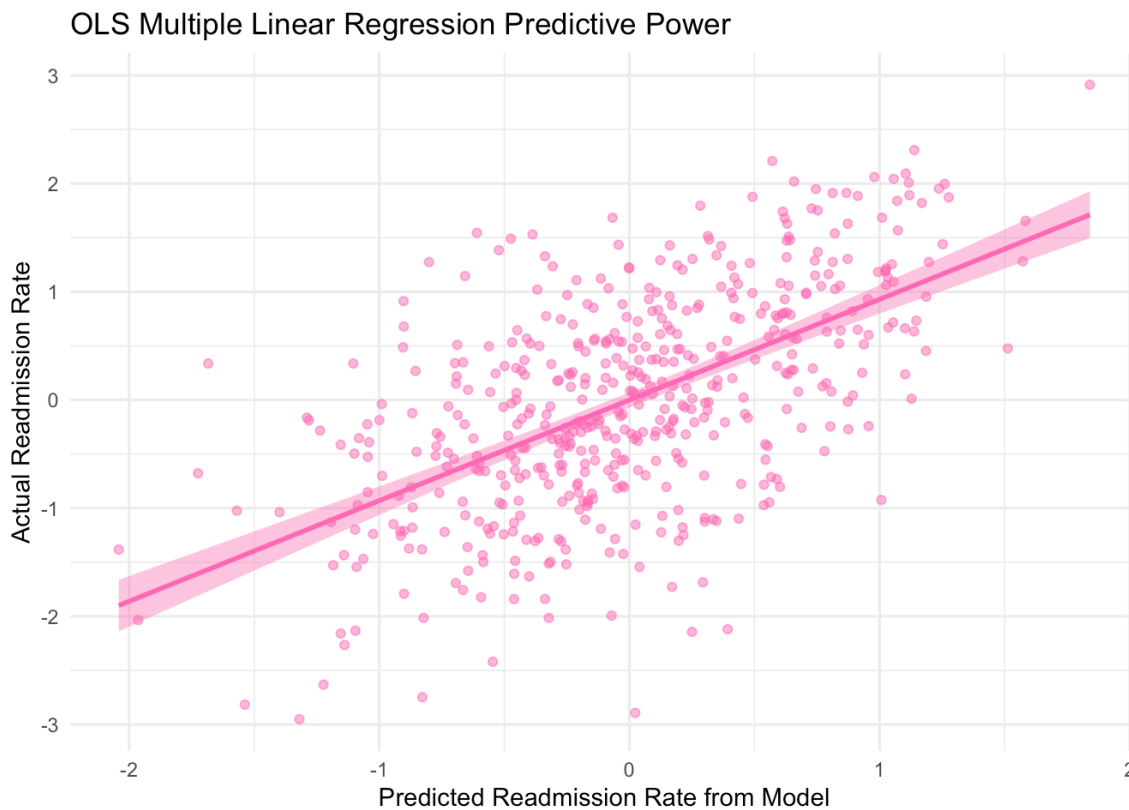


Figure 18. Predictions Plot for OLS Regression

NOTE that, while there is an obvious linear trend, it is weak. There is a lot of error around the predictions from the OLS regression!

6.2 The Elastic Net

Perhaps the seemingly low R^2 is because we still have too much multicollinearity or overfitting. So, as we discussed in lecture, we can address this using regularized regressions, such as the **Elastic Net**, which is more robust to multicollinearity by assessing penalties for non-zero variance. Typically, we would likely conduct a Ridge and/or LASSO before running an Elastic Net, but since we already know that (1) we have a fairly high amount of multicollinearity in our original `readyTrain` dataset and (2) we know we want to be able to perform feature reduction and figure out which features are important predictors of hospital readmissions, we're going to jump straight into Elastic Net, which performs both.

6.2.1 The Elastic Net Unpacked

In general, regularization is a technique that applies a penalty term to a cost function of a machine learning model, like the OLS regression. The reason is to discourage overfitting. This penalty term constrains the model's coefficients, which limits how flexible they are during training. Why?

The more flexible the coefficients, the less likely they will "learn" new data, i.e., the more generalizable they are! Thus, by applying this penalty (constraint), it improves the model's performance on "unseen" or new data.

As mentioned in lecture, Elastic Net regression is a general regularization technique that combines the regularization techniques of Ridge and LASSO to help us perform both feature selection (i.e., significant coefficients) and feature reduction (importance). The elastic net can even help us with the $P \gg n$ problem! Elastic net has two regularization terms, called **L1** (or the Lasso regularization term) and **L2** (the Ridge term).

- **L1**: makes some of the coefficients zero, thereby selecting only the most important features (shrinks them to zero). It is often said that this **encourages sparsity in the model**.

- **L2:** reduces the coefficients to small but non-zero values, thereby reducing the coefficients of the unimportant features (which we can use to determine significance). It does this by adding a penalty term to the cost function of the model, which is proportional to the coefficient squared. This allows us to retain all the features in the model but reduces the overall impact of the non-significant or less-important ones.

Thus, by combining these techniques together, the Elastic Net is balancing feature selection with feature reduction! If you're interested in reading the original paper on Elastic Net by Zou and Hastie (2005), you can find a free PDF of the paper here (<https://www.stat.purdue.edu/~tlzhang/mathstat/ElasticNet.pdf>).

6.2.2 Running, tuning, & cross-validating the Elastic Net

The elastic net can be run using the `glmnet` package (which we are accessing via `caret`), and can be cross-validated as random forest and SVM from our last project could be. This is a big improvement over OLS regression, which cannot be tuned. Additionally, the elastic net has **two hyperparameters** that we can tune, α and λ (not to be confused with the λ from Box-Cox transformation - Greek letters are abundant in statistics! Why do you think I named our course teams as I did?!).

- α is the strength of the regularization, representing the balance between **L1** and **L2** regularization. Larger α results in L1 regularization (feature selection / Ridge) whereas smaller α results in more L2 regularization (higher shrinkage / feature reduction / Lasso). When $\alpha = 0$ the regression is equivalent to OLS regression! α is also referred to as the **mixing percent** of L1 and L2 regularization.
- λ is the shrinkage parameter, such that when $\lambda = 0$ no shrinkage is performed and is equivalent to an OLS regression. As λ increases, the coefficients are increasingly shrunk and thus the more features will have coefficients shrunk to zero, regardless of the value of α .

6.2.2.1 We will start by setting our CV parameters just as we did on the last project, using the `trainControl()` function in `caret`.

NOTE that I am performing a 5-fold repeated cross-validation (CV) with 5 iterations.

```
ctrl <- trainControl(method = "repeatedcv", ## Do repeated CV
                    number = 5,           ## Number of k-folds
                    repeats = 5,          ## Number of repeats
                    search = "grid",       ## Grid search (vs. random)
                    verboseIter = FALSE)   ## Don't show me all the results
```

6.2.2.2 Then, we fit the model using the `train()` function in `caret`, specifying that we want to perform an elastic net, which will tune the model and perform CV per the specifications in the `trainControl()`. **NOTE** that I am allowing the values of α to range from $0 \rightarrow 1$ and allowing λ to range from $0 \rightarrow 5$.

```
## Create a search grid that allows alpha to range from 0 --> and
## allows lambda to range from 0 --> 5
searchGrid <- expand.grid(.alpha = seq(0, 1, length.out = 10),
                        .lambda = seq(0, 5, length.out = 15))

## Fit the elastic net using the tuning grid and CV scheme laid out
elasticMod <- train(bc_PredictedReadmissionRate ~ .,
                  data = readyTrain,
                  method = "glmnet",
                  tuneGrid = searchGrid,
                  trControl = ctrl)
```

Question 28: [1 point]

Explain what we are doing here. You may want to look up the parameters of the `trainControl()` and `train()` functions. Make sure to address what type of CV and hyperparameter search is being performed, as well as if both the α and λ hyperparameters of elastic net are being tuned.

Hint 1: Why are we doing cross-validation, and specifically why did I choose the one I did?

Hint 2: What does $\lambda = 0$ mean? How about $\alpha = 0$?

Your answer here. ##In this step, we are using repeated 5-fold cross-validation to tune our Elastic Net model and make sure the results are not based on one lucky data split. Instead of training and testing the model only once, the training data is divided into five groups (folds), where the model trains on part of the data and tests on the remaining portion. This process is repeated across all folds and then repeated five times to get a more stable estimate of performance. We are doing this because one train/test split can sometimes give misleading results, so repeated cross-validation helps us better understand how the model may perform on unseen data. We are also tuning the model's hyperparameters using a grid search. Rather than randomly choosing values, the model systematically tests combinations of alpha and lambda to see which performs best. The alpha parameter controls the balance between Ridge and LASSO regularization. When $\alpha = 0$, the model behaves like Ridge regression, while $\alpha = 1$ behaves more like LASSO, and values in between create an Elastic Net mix of both approaches. The lambda parameter controls how much regularization or shrinkage is applied to the coefficients. When $\lambda = 0$, no shrinkage occurs, meaning the model behaves similarly to an OLS regression, while larger values apply a stronger penalty to reduce overfitting.

6.2.3 Elastic Net Results

6.2.3.1 We are going to first take a look at the top 15 results of the Elastic Net in a table form, sorted from lowest *RMSE*:

Table 11. Top Performing Results of the Elastic Net Tuning & 5-fold Repeated Cross-Validation

alpha	lambda	RMSE	Rsquared	MAE	RMSESD	RsquaredSD	MAESD
0.00	0.00	0.78	0.39	0.62	0.02	0.04	0.02
0.11	0.00	0.78	0.39	0.62	0.02	0.04	0.02
0.22	0.00	0.78	0.39	0.62	0.02	0.04	0.02
0.33	0.00	0.78	0.39	0.62	0.02	0.04	0.02
0.56	0.00	0.78	0.39	0.62	0.02	0.04	0.02
0.44	0.00	0.78	0.39	0.62	0.02	0.04	0.02
0.67	0.00	0.78	0.39	0.62	0.02	0.04	0.02
0.78	0.00	0.78	0.39	0.62	0.02	0.04	0.02
1.00	0.00	0.78	0.39	0.62	0.02	0.04	0.02
0.89	0.00	0.78	0.39	0.62	0.02	0.04	0.02
0.00	0.36	0.79	0.39	0.62	0.02	0.04	0.02

alpha	lambda	RMSE	Rsquared	MAE	RMSESD	RsquaredSD	MAESD
0.11	2.14	0.94	0.32	0.74	0.03	0.03	0.02
0.11	2.50	0.96	0.31	0.76	0.03	0.03	0.02
0.67	0.71	0.99	0.25	0.78	0.03	0.03	0.02
0.44	1.07	0.99	0.25	0.79	0.03	0.03	0.02

NOTE: Folds that fail to converge will have an $R^2 = \text{NA}$.

6.2.3.2 We can also extract the hyperparameters of our best tuned model:

α :

```
## [1] 0
```

λ :

```
## [1] 0
```

Question 29: [1 point]

Check the values of α and λ here. What do they suggest about the optimal regularization that is being fit here? (Recall that when both α and λ are zero, it's an OLS regression!)

Your answer here. ##The best tuning values for the Elastic Net model were alpha = 0 and lambda = 0. Since alpha = 0 represents Ridge regression and lambda = 0 means no penalty is being applied, the model ended up behaving like an OLS regression. Even though Elastic Net tested different combinations of penalties, the best-performing version behaved essentially like an OLS regression. This suggests that adding regularization did not noticeably improve model performance for this dataset, which also makes sense given that the OLS and Elastic Net performance metrics were very similar.

Let's also plot the results of the tuning search. Can you figure out the type of tuning search you did? (**Hint:** the visual pattern should confirm your answer from earlier!) The diamond denotes the best combination of hyperparameters!

```
results <- elasticMod$results %>%
  data.frame()
best <- results[as.numeric(rownames(elasticMod$bestTune)), ]
```

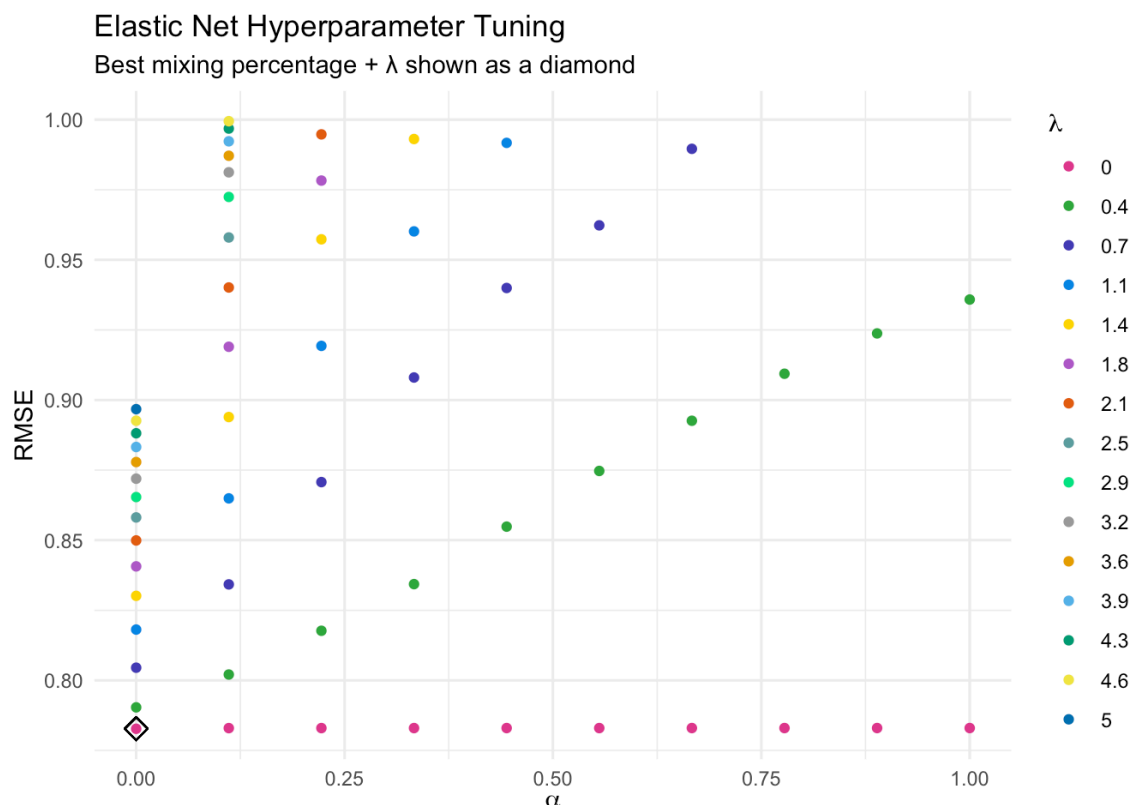


Figure 19. Hyperparameter Grid Search for Elastic Net

6.2.3.3 Important Features

Important features can also be extracted, using the `varImp()` function from `caret`.

```
important <- varImp(elasticMod)$importance
```

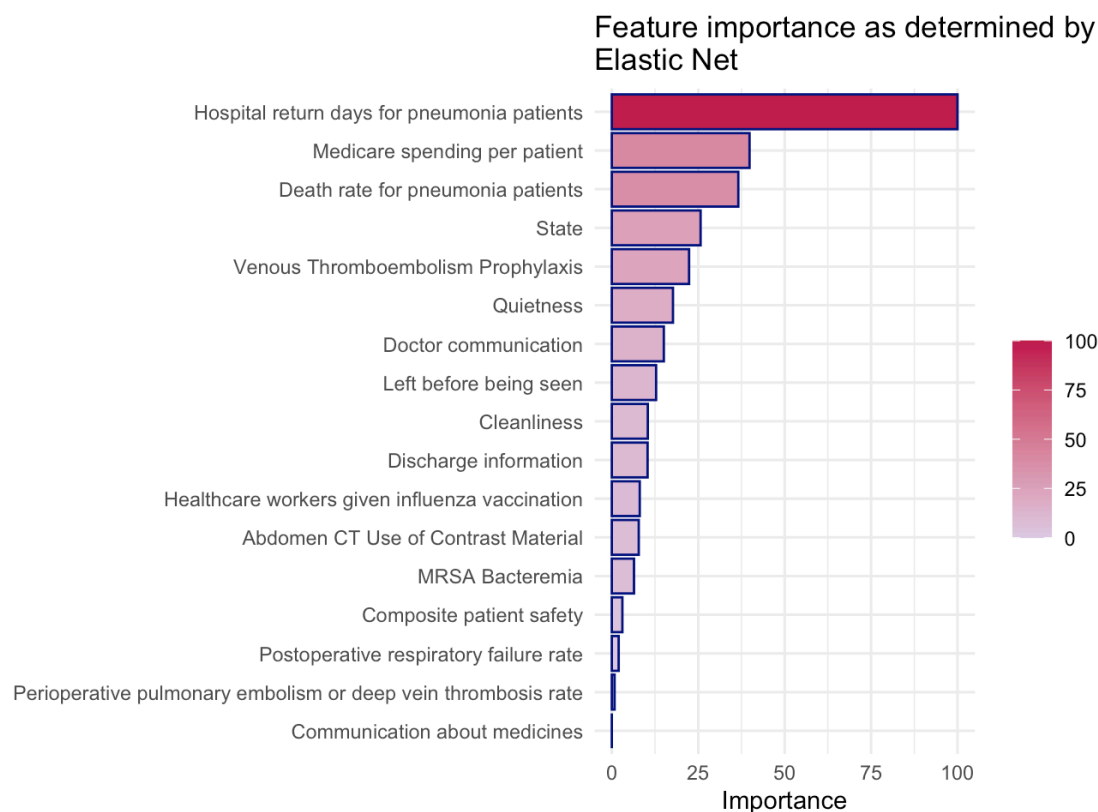


Figure 20. Variable Importance from Best Elastic Net Model

Question 30: [1 point]

Which of the features were most important? How does this compare to the most important features out of OLS regression? Make sure you're looking at the **absolute value of the beta coefficients to assess importance in OLS regression**.

Your answer here. ##The most important features in the Elastic Net model were hospital return days for pneumonia patients, Medicare spending per beneficiary, death rate for pneumonia patients, state, thromboembolism prophylaxis, and quietness, since these had the highest importance scores. Hospital return days for pneumonia patients stood out the most by far, while variables such as doctor communication, cleanliness, discharge information, and time before being seen had smaller but still noticeable importance. Variables like communication about medicines and postoperative respiratory failure contributed very little to prediction. Compared to OLS, the overall pattern looked fairly similar, especially for pneumonia-related outcomes and hospital quality measures. This makes sense because the Elastic Net model ended up with $\alpha = 0$ and $\lambda = 0$, meaning it behaved very similarly to an OLS regression. When comparing importance in OLS, we focused on the absolute value of the beta coefficients, since larger coefficients indicate greater importance regardless of whether the relationship is positive or negative.

How about about of PCA? You may have already noticed that importance almost seems inverted! Why?!

Hint 1: What is the fundamental difference between a PCA and any regression?

Hint 2: What does "importance" mean in a PCA vs. any regression?

Hint 3: What does the sign of the loadings mean in a PCA? Are they actual or arbitrary?

Your answer here. ##PCA looked different because it was trying to answer a different question than Elastic Net or OLS. In our results, variables such as hospital return days for pneumonia patients, Medicare spending per beneficiary, and death rate for pneumonia patients were among the strongest predictors in Elastic Net because they helped predict pneumonia-related outcomes. However, PCA focuses on identifying variables that explain the most variation across hospitals overall, not necessarily variables that are best at prediction. Because of this, some variables that looked highly important in Elastic Net or OLS may not appear as important in PCA, while variables with more overall variation across hospitals may stand out more. The importance can also seem inverted because PCA loadings can flip signs without changing meaning, so what matters more is the size of the loading rather than whether it is positive or negative.

6.2.3.2 Check model performance

Assessing **model performance** means assessing how well our training model does when applied to "unseen" data, here our `readyTest` dataset. Because we know the right answer - the true value of `PredictedReadmissionRate`, we can assess the error that the training model produces when fitted against the testing data. We measure that error as *RMSE* and *MAE*. We briefly reviewed what the *RMSE* measures back in Question 36, so I will only review what the *MAE* is here.

Mean Absolute Error (*MAE*) is the average of the absolute differences between predicted and actual values, such that $MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$ where y_i is each observed value, $\hat{y}_i$ is each corresponding predicted value, and n is the number of observations. So, unlike *RMSE*, *MAE* treats all errors equally and therefore is more robust to outliers. So, *MAE* will tell you the average magnitude of the error, regardless of direction!

Let's also make predictions on the testing sets for both the OLS and Elastic Net models using the `postResample()` function in `caret`; this will not only do predictions, but it will enable us to more easily generate metrics that we can use to compare model performance, such as R^2 and *MAE*.

```
## Post-resample on OLS
prOLS <- postResample(pred = predict(bestMod, newdata = readyTest),
                      obs = readyTest$bc_PredictedReadmissionRate)

## Post-resample on Elastic Net
prElastic <- postResample(pred = predict(elasticMod, newdata = readyTest),
                          obs = readyTest$bc_PredictedReadmissionRate)
```

Table 12. Comparison of Model (Test) Performance of OLS and Elastic Net Regressions

	RMSE	Rsquared	MAE
prOLS	0.81	0.34	0.65
prElastic	0.81	0.35	0.64

6.2.3.3 Check for overfitting/underfitting

Using the same method, check for overfitting / underfitting. Make predictions using `readyTrain` rather than `readyTest`, and then check how the *RMSE* on the training data compares to the *RMSE* of the training data for the Elastic Net. How do they compare? **How do both compare to what we got out of the OLS regression?**

```
## Post-resample on OLS
prOLS <- postResample(pred = predict(bestMod, newdata = readyTrain),
                      obs = readyTrain$bc_PredictedReadmissionRate)

## Post-resample on Elastic Net
prElastic <- postResample(pred = predict(elasticMod, newdata = readyTrain),
                          obs = readyTrain$bc_PredictedReadmissionRate)
```

Table 13. Comparison of Training Performance of OLS and Elastic Net Regressions

	RMSE	Rsquared	MAE
prOLS	0.78	0.4	0.61
prElastic	0.78	0.4	0.61

TAKEAWAY

When it comes to which performed better, it's obvious that the Elastic Net only marginally (very marginally!) out-performed the OLS regression or the Elastic Net (Table 12). The R^2 is a 0.01 higher in the Elastic Net model and both measures of error are a teensy bit lower. But, truthfully, they're performing the same **because they are the same model**. The Elastic Net likely out-performed simply because it was cross-validated!

To interpret $RMSE$ from training (Table 13) vs. testing (Table 12), we're looking first to see if they are the same/balanced. However, we can see that error ($RMSE$, MAE) is a bit higher in the testing post-resample vs. the training post-resample. While this could suggest that we have some mild overfitting (our model memorized the training data a little too well leading to poor generalization when it "saw" the testing data), instead I think it is more likely is that we have **underfitting** due to a lack of complexity! Underfitting error patterns are such that you will see roughly the same error in both the testing and training sets, and both error rates are quite high.

6.2.3.4 Graphical comparison of the two models' predictions

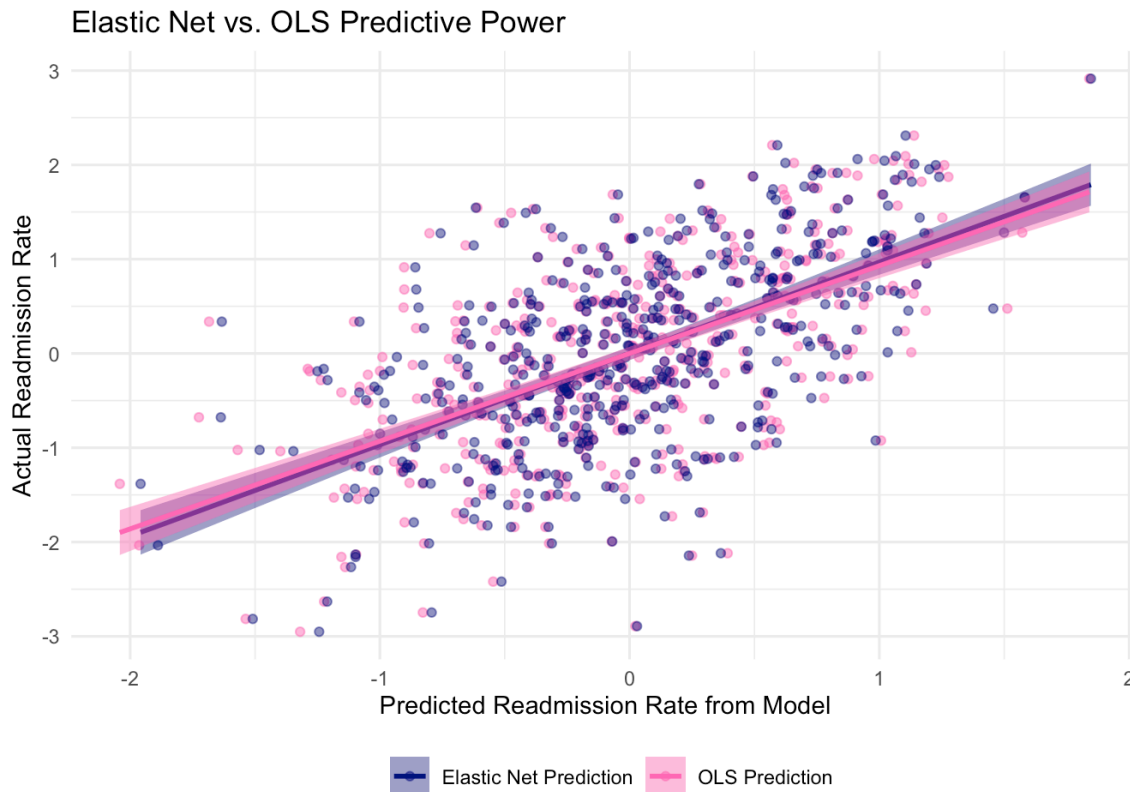


Figure 19. Predictions Plot for Elastic Net vs. OLS Regression

It should not come as a surprise, but these models are performing **VERY** similarly!

Question 32: [4 points]

What conclusions do you make at this point about whether to make predictions with an OLS regression, Elastic Net, or neither? What recommendations would you make to our client at this point? What other analyses might you suggest, or other data we might want to include? Please try to flesh out 2-3 recommendations in at least a paragraph.

HINT: Make sure to ask yourself whether these data appear linear or not. Could non-linearity be a cause for a low R^2 ? If not, what other causes are there and how will that impact your recommendations?

Your answer here. ##At this point, Team Alpha would probably lean toward using an OLS regression over Elastic Net if predictions needed to be made, mainly because the two models performed almost identically while OLS is easier to interpret and explain to healthcare stakeholders. Elastic Net only performed marginally better, and since it selected $\alpha = 0$ and $\lambda = 0$, it essentially behaved like the same model. At the same time, we would still be somewhat cautious about relying too heavily on predictions from either model because the R^2 values suggest the models explain only a moderate amount of variation in pneumonia-related outcomes.

Based on the residual plots and prediction graphs, the data still appear to follow a mostly linear pattern, so nonlinearity does not seem like the main reason performance is limited. Instead, one possible explanation is that important predictors may be missing from the dataset or not fully captured. We also know that a meaningful portion of the data required imputation or had missing values earlier in the analysis, which may have contributed to lower predictive performance and reduced model accuracy. Since pneumonia-related outcomes are likely influenced by many factors, it is also possible that variables such as patient severity, staffing levels, hospital resources, demographics, or community-level health factors are playing a role that our current dataset is not fully capturing.

As a recommendation to the client, we would suggest continuing with a simpler and more interpretable model such as OLS for now, while also improving the available data and testing additional predictors in future analyses. We would also recommend exploring more flexible modeling approaches to see whether more complex relationships exist that linear regression may not be capturing.

7 Choose Your Own Adventure

It's time to choose your adventure! After working through this demo and answering the 31 questions, you will choose one of three possible trajectories. The following are some brief hints or tips for each of the choices available to you.

Adventure 1.

[Choosing one other condition to analyze.]

- Make sure that you use the code I provided, updating it for your new condition
- Make sure to answer all of the interpretative questions again for your new dataset.
- I will specifically be looking for a comparison of which condition, pneumonia or the one you chose, seems to be a better choice for a predictive model for our client. Or is it neither?
- Make sure to include some next steps or recommendations based on your new analysis!

Recommendations & Hints:

You can avoid going back to Demo 1 to do your importing, cleaning, and pre-processing by running this source code:

```
## Make any necessary changes in the R source code
## Uncomment to run!
# source(file = "reRunDemoData.R")
```

- Make sure to update your filepath to where you stored the hospital files on YOUR local machine, just as you did in Question 1 of the Demo!
- Make sure to update the condition that gets selected from "PN" to the condition of your choosing. A full list of available conditions is found in Demo 1.

Adventure 2.

[Choosing two or more conditions to analyze.]

- Choose a slightly more complicated analysis to undertake, for example, focusing on surgical interventions (HIP-KNEE & CABG) or heart-related conditions (HF, AMI, & CABG).
- Make sure to note that the readmissions rates are **averages** across the total conditions you've selected!
- See the same recommendations and hints as Adventure 1.

Adventure 3.

[Continue analyzing the pneumonia data.]

- Make sure to justify your choice.
- Choose at least two new algorithms to perform. They can be unsupervised or supervised, but must be 2+.
- A natural extension would be to do a Ridge and/or a LASSO; the former would tell you if you should be controlling for multicollinearity more than you currently are, the latter would tell you if you need to further shrink coefficients than you currently are. Additionally, you could consider a regression-based method that incorporates **non-linearity**, such as a random forest. However, you are free to choose anything you think is appropriate here!
- Other choices I think you could potentially justify to stakeholders, if you were interested:
 1. Switch to classification-based random forest using
`ComparedToNational_Hospital` return days for pneumonia patients
 2. kNN using the 2 clusters we've identified and named in our segmentation analysis
 3. Our data are also ready for a neural network analysis (e.g., a feed-forward NN)!

Deliverables

I am looking for 1-2 markdown documents with their knitted HTML. For example, if you're choosing Adventures 1 or 2, you may want to work through this document once, make a copy of this markdown document, move the source code to the **TOP**, edit the source code as needed, and then go through this again from the beginning using the new condition(s) you chose.

Alternatively, if you're choosing Adventure 3, maybe you just choose to add on to the bottom of this document, replacing the 'Choose Your Own Adventure' section with your actual adventure.

Just make sure to answer the questions well, and make sure to justify the decisions you make. Tell me **WHY** you're choosing the condition(s) you are in Adventures 1 or 2, or Tell me **WHY** you're doing the analyses you are in Adventure 3. Other than that, make this your own learning adventure!

References

- Box, G. E. P., and D. R. Cox. 2018. "An Analysis of Transformations." *Journal of the Royal Statistical Society: Series B (Methodological)* 26 (2): 211–43. <https://doi.org/10.1111/j.2517-6161.1964.tb00553.x> (<https://doi.org/10.1111/j.2517-6161.1964.tb00553.x>).
- De Alba, Israel, and Alpesh Amin. 2014. "Pneumonia Readmissions: Risk Factors and Implications." *Ochsner Journal* 14 (4): 649–54.
- R Core Team. 2019. *R: A Language and Environment for Statistical Computing*. Vienna, Austria: R Foundation for Statistical Computing. <https://www.R-project.org> (<https://www.R-project.org>).
- Yeo, In-Kwon, and Richard A. Johnson. 2000. "A New Family of Power Transformations to Improve Normality or Symmetry." *Biometrika* 87 (4): 954–59. <https://doi.org/10.1093/biomet/87.4.954> (<https://doi.org/10.1093/biomet/87.4.954>).
- Zou, Hui, and Trevor Hastie. 2005. "Regularization and Variable Selection via the Elastic Net." *Journal of the Royal Statistical Society Series B: Statistical Methodology* 67 (2): 301–20. <https://doi.org/10.1111/j.1467-9868.2005.00503.x> (<https://doi.org/10.1111/j.1467-9868.2005.00503.x>).